

From Firmware to Screen

Real-Time Control, Simulation, and
Visualization in C++23 With CUDA
and Unreal Engine

KEVIN GOMEZ

SECTION 1

The problem

Separate tools for scheduling, simulation, operations and visualization

A real-time system is usually four tools stitched together

A typical embedded project builds its system from separate tools. Each one is good at its job, and each one has its own model of the world.

Job	Usually done with	Which brings its own
Scheduling	an RTOS on the microcontroller; threads or a middleware executive on Linux	task model, timing, configuration
Simulation	a separate modelling tool or simulator	time base, data types, build
Operations	a hand-built ground console or a folder of scripts	command and telemetry definitions
Visualization	a game engine or plotting tool, glued on at the end	message format, frame rate

Every boundary between two tools is a translation layer that someone has to write, and keep in sync.

The bugs live in the seams

- ▶ **The algorithm is written twice**: once for the simulator, again for the target. The two copies drift.
- ▶ **The data is defined three times**: a telemetry field is a struct in firmware, a column in the console and a message in the visualizer. Change one, break the others.
- ▶ **Time means something different in each tool**: simulation steps, RTOS ticks, wall-clock time, render frames.
- ▶ **Tests stop at the tool boundary**: each piece passes on its own, and the integrated system first runs on hardware, late.
- ▶ **Every new engineer learns four ecosystems** before they change one line.

The idea: one runtime architecture

Build every piece as a **component** on one shared C++ runtime, and let the tools attach to the running application through standard interfaces.

What you write	Component type	For example
A physics or environment model	SW_MODEL	vehicle dynamics, atmosphere, terrain
An emulated device	HW_MODEL	a sensor that produces what the real one would
An interface to real hardware	DRIVER	a serial link to a flight controller
A service the system needs	SUPPORT	telemetry, a bridge to a 3D viewer

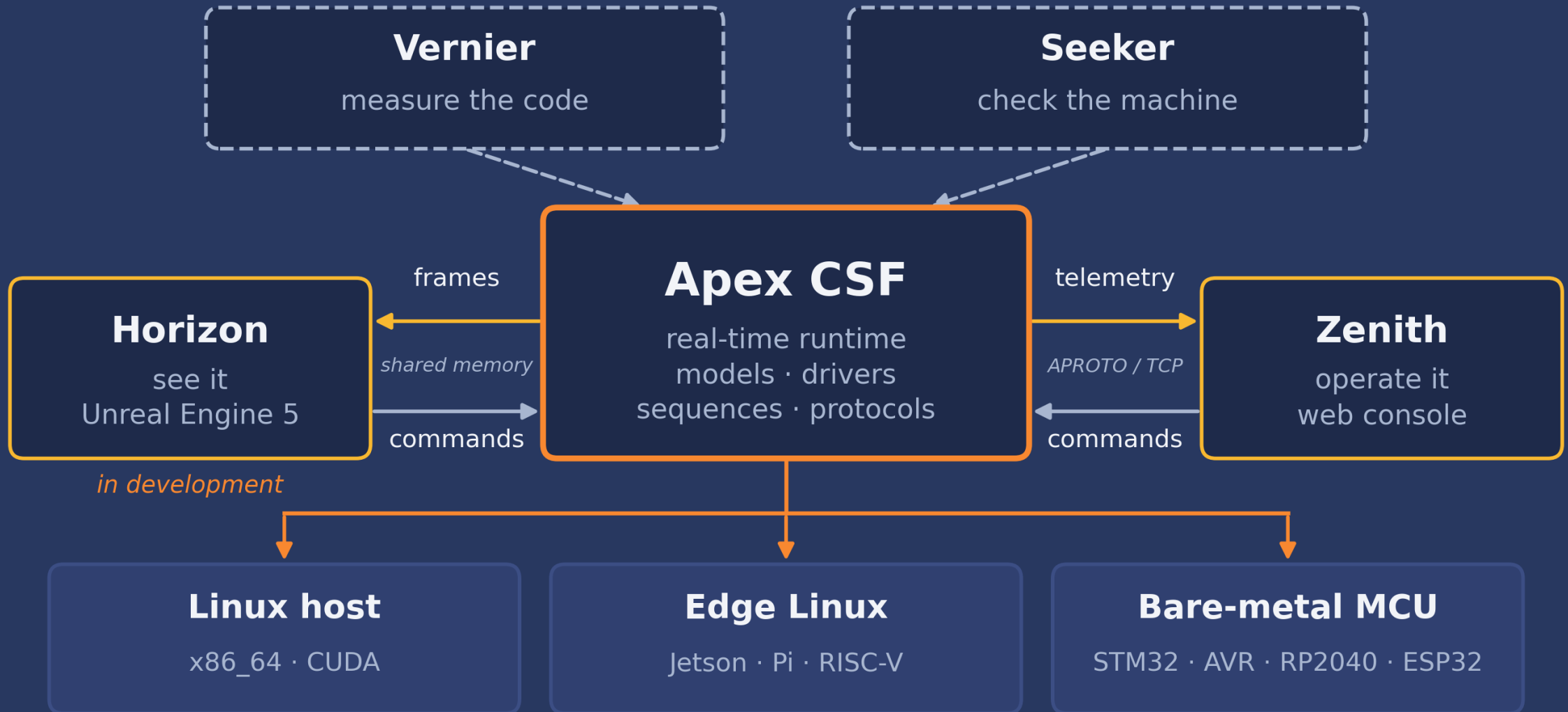
- ▶ Every component gets the same lifecycle, scheduling, parameters, commands and telemetry.
- ▶ Operations and visualization are **clients** of the running application, not separate systems with their own copy of the data.

What "one codebase" covers

Tier	Targets	What runs there
Linux host	x86_64, with or without CUDA	the full executive: multi-threaded scheduler, runtime parameters, library hot-swap, protocols, Monte Carlo
Edge Linux	NVIDIA Jetson, Raspberry Pi, RISC-V 64	the same executive, cross-compiled
Bare-metal MCU	STM32, AVR, RP2040, ESP32	a zero-heap MCU executive and scheduler on the same component core

- ▶ 21 core libraries carry a support contract: each builds at C++17, 20 and 23 on Linux and on all four MCU families.
- ▶ One Docker image per platform, one command per build: `make compose-jetson-release`, `make compose-stm32`.

The stack



Why C++ for Apex CSF

- ▶ **One language from an 8-bit AVR to a GPU**: the same source builds with Clang 21 on Linux, arm-none-eabi GCC for STM32 and Pico, avr-gcc, and ESP-IDF's GCC, with CUDA kernels in the same tree.
- ▶ **Abstractions that cost nothing at runtime**: schedulers sized by template parameters, and tasks bound as member-function delegates, with no heap and no `std::function`.
- ▶ **Deterministic by construction**: no exceptions or RTTI on microcontrollers, status codes instead of throws, fixed capacity on the hot path.
- ▶ **Firmware code tested on the host**: header-only math, checksums and framing compile for both, so host tests exercise the code the boards run.
- ▶ **It's where the tools already are**: CUDA, Unreal Engine, Google Test, the sanitizers and clang-tidy all speak C++.

How Apex CSF uses C++

Paradigm	Where in Apex CSF	Why
Non-virtual interface	<code>init()</code> runs fixed steps, then the component's <code>doInit()</code>	one lifecycle, host or board
Virtual interfaces	<code>IUart</code> , <code>ICan</code> , <code>IFlash</code> , with <code>final</code> board classes	change the board, keep the app
Templates, CRTP	<code>McuScheduler<MaxTasks></code> , cipher base classes	sized and dispatched at compile time
Member delegates	<code>registerTask<T, &T::step></code>	a task is two pointers, no heap
RAII	file descriptors, OpenSSL handles, CUDA buffers	nothing leaks on an early return
noexcept, status codes	the whole API; no exceptions or RTTI on MCUs	predictable control flow
static_assert	wire-struct layouts, generated headers	layout mistakes fail the build
Lock-free, seqlock	MPMC queue, parameter bank	real-time readers never block

Standards: the host builds as C++23; the portable core is written to C++17 and picks up newer features like `std::span` and `std::byteswap` through feature-detection shims.

Apex CSF: the runtime

A C++23 real-time component framework. You write components; it handles scheduling, lifecycle, communication, logging and deployment.

- ▶ **One component model** from Linux hosts and NVIDIA Jetson down to bare-metal STM32, AVR, RP2040 and ESP32.
- ▶ **Real-time executive**: parameters reloaded at runtime and verified before they apply; component libraries swapped while running (Linux).
- ▶ **Lock-free task dispatch and logging**, with an allocation-free hot path by design.
- ▶ **Hardware layer**: common UART, CAN, SPI, I2C, flash and PPS interfaces over four MCU families.
- ▶ **Protocols**: UART, SPI, I2C, CAN, LIN, Modbus, TCP, UDP, Bluetooth, CCSDS space packets, APROTO for commands and telemetry, and SLIP and COBS framing (a serial line is just a stream of bytes; framing marks where each packet ends).
- ▶ **More than a scheduler**: sequences and watchpoints, a time server, Monte Carlo runs, simulation libraries and CUDA compute.
- ▶ **Built and tested the same way everywhere**: the hardware layer is mocked on the host, with sanitizers and static analysis on every pull request.

Zenith: operate it

A web operations console. Connect to a running application, send commands, watch telemetry and tune parameters.

- ▶ **Configuration-driven**: each target is a folder of generated dictionaries for data structures, commands and plot layouts. No component knowledge is hard-coded.
- ▶ **Protocol-neutral**: APROTO for Apex CSF, CCSDS space packets for NASA cFS, over TCP or UDP.
- ▶ **Operate**: typed command forms, parameter upload with verify-then-apply, sequence upload checked against the vehicle's own rules, live library swap, file transfer.
- ▶ **Watch**: live telemetry over WebSocket, strip charts, stored history with downsampled envelopes, CSV and PNG export.
- ▶ **Many targets at once**, each with its own connection, storage and reconnect; optional login and an audit log.

Horizon: see it

Connects a running simulation to Unreal Engine 5 for a live 3D view.

- ▶ **Separate processes**: the simulator and Unreal share only a small bridge library.
- ▶ **Shared-memory rings in both directions**: state frames out at the simulation's rate, operator commands back.
- ▶ **Procedural geometry kit**: vehicles, wings and scenes generated from code. Header-only, with no binary assets.
- ▶ **Shared world data**: terrain tiles, atmosphere tables and gravity fields read by both sides, and checked by hash so they agree.
- ▶ **Demos**: a lidar test box, airfoil and airplane wind tunnels, a rover on terrain, an aircraft in a standard atmosphere.
- ▶ Unreal Engine 5.7 plugins, C++20, Linux.

In development, and not public yet.

Vernier: measure the code

Performance tests that are Google Test tests, measured with statistics and checked in CI.

- ▶ **PERF_TEST is a Google Test test**: every result reports median, percentiles and spread, and flags unstable measurements.
- ▶ **15 profilers behind one flag**, including perf, callgrind, heaptrack, bpftrace, Intel RAPL energy, and NVIDIA Nsight Systems and Nsight Compute.
- ▶ **GPU**: CUDA kernels timed in the test process, multi-GPU and unified memory, with NVTX ranges added for you.
- ▶ **CI gating**: `bench compare --fail-on-regression` fails a pull request that makes the code slower.
- ▶ A Rust `bench` command-line tool and Python plotting tools.
- ▶ Linux on x86_64 (with or without CUDA), Jetson, Raspberry Pi and RISC-V; the core harness also builds on macOS.

Seeker: check the machine

Tells you whether a Linux machine is set up for real-time work, and if not, what's in the way.

- ▶ **57 modules in 8 domains**: CPU, memory, storage, network, timing, devices, system and GPU.
- ▶ **34 command-line tools**, including an `rtcheck` validator for each domain that prints PASS, WARN, FAIL or SKIP with the fix.
- ▶ **Made for real-time callers**: every API is `noexcept`, every function is marked RT-safe or not, and results are fixed-size.
- ▶ **Read-only**: it never changes the system and doesn't need root.
- ▶ **For CI and startup checks**: exit codes and JSON output, or link one domain with `find_package(seeker)`.
- ▶ Linux on x86_64, Jetson, Raspberry Pi and RISC-V.

SECTION 2

Framework architecture

One component model, then the same model on Linux and on microcontrollers

Everything is a component

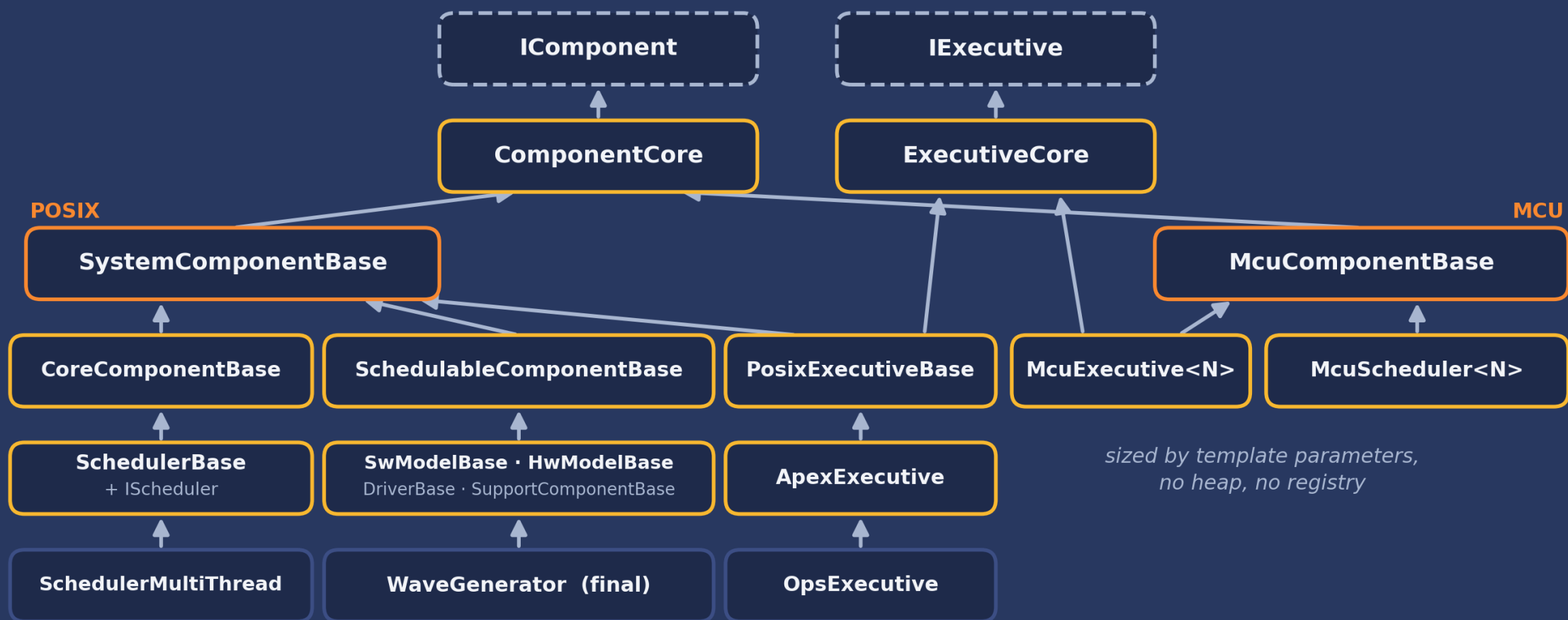
- ▶ IComponent **is the contract**: identity (componentId, componentName, componentType, label), init, reset and status. It includes only `<stdint.h>`, so it compiles on bare metal.
- ▶ ComponentCore **fixes the lifecycle**: `init()` always runs `preInitCheck()`, then `preInit()`, then the component's own `doInit()`.
- ▶ **Every instance has a fullUid**: `componentId << 8 | instance`, so the two WaveGenerators are `0xD000` and `0xD001`.
- ▶ **Every component has a type**, and the executive acts on it. The next slide defines them.

Component types

Type	What it is	Who writes it	Examples
EXECUTIVE	the orchestrator; one per app	framework; your app subclasses it	OpsExecutive, McuExecutive
CORE	a framework service the executive creates itself; one of each	framework	scheduler, registry, interface, action engine
SW_MODEL	simulates something: physics, environment, compute	you	WaveGenerator, GPU models
HW_MODEL	stands in for a device, over a virtual port	you	an emulated flight controller
DRIVER	talks to real hardware	you	the HIL serial driver
SUPPORT	a runtime service an app opts into	framework or you	SystemMonitor, DataTransform

- ▶ **Executive and core components run the framework** and have no scheduled tasks of their own. The other four register tasks and get the internal message bus.

The component hierarchy



dashed box: pure interface · arrow points to the base · two arrows out of one box: multiple inheritance
the executive joins two hierarchies with no diamond: ExecutiveCore is not an IComponent

The build

- ▶ **CMake presets and Ninja**: one preset per target, hosted, cross-Linux or bare metal, each with its toolchain file.
- ▶ `apex_add_library` **and friends**: a library's CMakeLists is a few lines; tests, precompiled headers and code generation hook in.
- ▶ **One Docker image per platform**, with its compiler, SDK and flash tools; Clang 21 on the host; ccache and mold keep rebuilds fast.
- ▶ **Make is the front door**: `make debug`, `make jetson-release`, `make compose-stm32-flash`. One table lists every platform.

CI

When	What runs
Every pull request	build and tests with CUDA, sanitizers, clang-tidy, format and tool tests
Nightly	thread and realtime sanitizers, cppcheck, coverage, security scans
On a tag	per-platform release builds, an SBOM and checksums

- **Scoped to the change and run in the same Docker images as a laptop:** a docs-only PR runs lint alone.

SECTION 2 · LINUX

Building an app on Linux

Running examples: the ops demo, the action demo and the hardware-in-the-loop rig

Running example: the ops demo

A software-in-the-loop app built to exercise the framework end to end: waveform generators produce telemetry, and every command path gets used. Most slides in this section point back at it.

Component	fullUid	Type
OpsExecutive , an ApexExecutive	0x000000	EXECUTIVE
Scheduler, FileSystem, Registry, Interface, Action engine	0x000100 to 0x000500	CORE
SystemMonitor, TelemetryManager	0x00C800, 0x00C900	SUPPORT
WaveGenerator , two instances	0x00D000, 0x00D001	SW_MODEL
TestPlugin , version 1 and version 2 as .so files	0x00FA00	SW_MODEL

- ▶ Runs on a laptop or a Raspberry Pi 4; Zenith connects to it over TCP.

ApexExecutive: the orchestrator

- ▶ **It owns time**: a clock thread on an absolute deadline grid drives every tick. Components never create threads.
- ▶ **It owns the task table**: every component's tasks, dispatched by the scheduler at the rates in TPRM.
- ▶ **It owns the lifecycle**: register each component, load its parameters, `init()`, freeze the registry, start the threads; a staged shutdown at the end.
- ▶ **It owns the command path**: health, pause, sleep, lock, parameter and library reloads, restart, inspect.

Thread	Job
Clock	ticks on the deadline grid and detects overruns
Task execution	hands each tick's tasks to the scheduler's worker pools
External I/O	commands in, telemetry out
Watchdog	checks the clock and task lag; heartbeat to a supervisor process
Startup, shutdown	the staged lifecycle

ApexExecutive: what an app writes, and what runs

```
class OpsExecutive : public executive::ApexExecutive {
public:
    using ApexExecutive::ApexExecutive;
protected:
    bool registerComponents() noexcept override {
        const auto& LOG_DIR = fileSystem().logDir();
        return registerComponent(&waveGen0_, LOG_DIR) &&
            registerComponent(&waveGen1_, LOG_DIR) &&
            registerComponent(&sysMonitor_, LOG_DIR);
    }
private:
    wave::WaveGenerator waveGen0_, waveGen1_;
    system_core::support::SystemMonitor sysMonitor_;
};
```

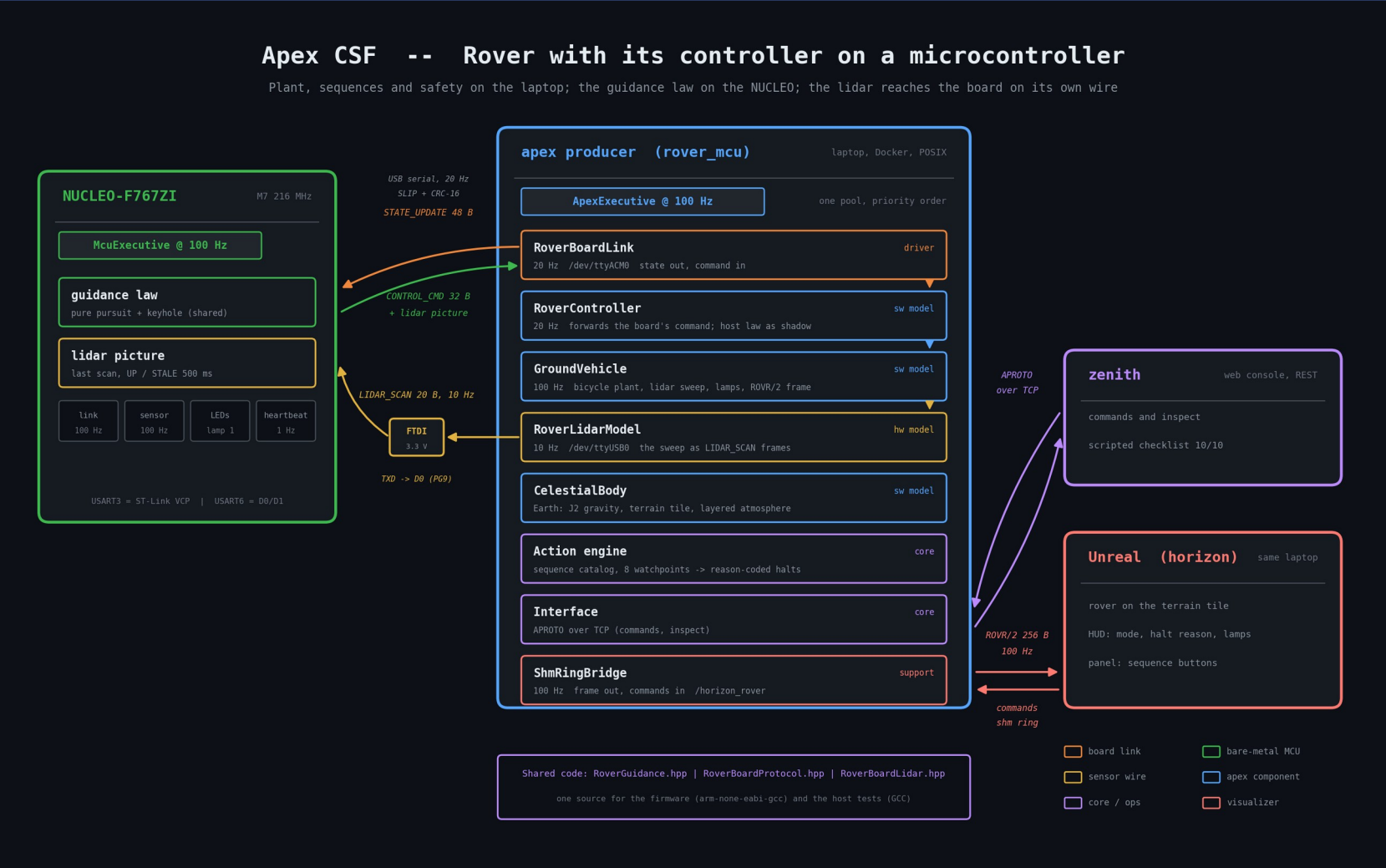
- ▶ **Components are members**: their lifetime is the executive's. No heap, no factories.
- ▶ **Everything else is TPRM**: task rates, thread priorities, action rules.
- ▶ **Real-time modes**: soft modes skip or log a busy tick; hard modes shut down on a violation.

Core components

Component	What it does	How
Scheduler	runs every task at a fraction of the base tick, alone or on worker pools	lock-free dispatch, checks before the clock starts
Registry	one lookup table for components, tasks and data blocks	register, then <code>freeze()</code> ; fixed storage after
FileSystem	owns the on-disk layout and atomic writes	A/B banks, a root lock, write then rename
Interface	the ground link for commands and telemetry, and the internal bus	APROTO over TCP or UDP on epoll, SLIP or COBS framing
Action engine	watches data and fires commands or sequences	watchpoints, events, RTS and ATS sequences
TimeServer	the node's source of UTC	PPS edges correlated to the steady clock

- Logging isn't a component: an asynchronous logger with a lock-free ring and one shared drain thread.

A little of everything: the rover demo's configuration



The scheduler and its variants

Variant	Runs on	How it runs a tick
SchedulerMultiThread	Linux, Jetson	hands due tasks to lock-free worker pools, by priority
SchedulerSingleThread	Linux	runs due tasks in order on one thread
McuScheduler<MaxTasks>	bare metal	walks a static table sorted by priority; no heap

```
# ops demo: WaveGenerator #0 waveStep @ 100 Hz
fullUid = { type = "uint", size = 4, value = 53248 }
freqN = { type = "uint", size = 2, value = 100 }
freqD = { type = "uint", size = 2, value = 1 }
offset = { type = "uint", size = 2, value = 0 }
priority = { type = "int", size = 1, value = 127 }
```

- **Every task is a fraction of one tick:** rate $\text{freqN} / \text{freqD}$, an offset to spread load, a priority from -128 to 127 .

Scheduler: dispatch without locks or allocation

- ▶ **Tasks are delegates**: a function pointer and a context, bound once. No `std::function`, no heap.
- ▶ **Dispatch** is a compare-and-swap push onto a bounded Vyukov MPMC ring; a futex wakes a worker only if one is parked.
- ▶ **A full ring is loud**: the task is dropped and logged, never silently delayed.
- ▶ **Per-task statistics** in atomics: last and worst runtime, completions, overruns, deadline violations, skips.
- ▶ **Allocation happens before the clock**: `addTask()` and `init()` may allocate; the tick path doesn't. RealtimeSanitizer checks it nightly.

Measured on an x86 host, 2026-08-29

Multi-threaded scheduler, empty tick	52 ns
Single-threaded scheduler, 100 no-op tasks	0.90 μ s
Lock-free enqueue against the old locked pool	8.3 $\times$ faster
MCU scheduler, empty tick	9 ns (host build)

TPRM: parameters from a spec, not from code

```
[WaveGenTunableParams]
frequency = { type = "float", size = 4, min = 0.0, max = 50.0, value = 1.0 }
amplitude = { type = "float", size = 4, min = 0.0, value = 1.0 }
dutyCycle = { type = "float", size = 4, min = 0.0, max = 1.0, value = 0.5 }
waveType = { type = "uint", size = 1, allowed = [0, 1, 2, 3, 4], value = 0 }
```

```
struct WaveGenTunableParams { // generated into .auto/, never edited by hand
    float frequency{1.0F}; float amplitude{1.0F}; float dcOffset{0.0F};
    float phaseOffset{0.0F}; float noiseAmplitude{0.0F}; float dutyCycle{0.5F};
    std::uint8_t waveType{0}; std::uint8_t reserved[3]{}; float reserved2{0.0F};
};
static_assert(sizeof(WaveGenTunableParams) == 32, "layout diverged from the spec");
```

- ▶ Out-of-range values never become a binary, and a payload built for a different layout is rejected when it loads.

Changing a live system

Command	What changes	How
RELOAD_TPRM	one component's parameters	load from the inactive bank, validate, swap
RELOAD_LIBRARY	one component's code	lock it, load the new <code>.so</code> , re-point its scheduler tasks, swap banks
RELOAD_EXECUTIVE	the whole binary	acknowledge, then <code>execv</code> ; the bank swap is undone if <code>execv</code> fails

- ▶ `ParamBank<T>` holds a staged and an active set. Publishing is an atomic swap; readers use a seqlock and never see a torn value; one level of rollback.
- ▶ **Check before you apply:** `READBACK` and `VERIFY` report on the staged bank without touching the running one.
- ▶ **Rejects say why:** a CRC, constraint or layout mismatch each report their own fault code, and the component keeps its current values.
- ▶ **Boot safety:** bad parameters at boot fall back to the other bank once, then a `SAFE` hold that ground can still command and fix.

Commanding: APROTO

Bytes	Field
0-1	magic, 'A' 'P'
2	version
3	flags: response, acknowledgement requested, CRC present
4-7	fullUid: which component
8-9	opcode
10-11	sequence number
12-13	payload length

- ▶ **A 14-byte header** and an optional CRC-32C trailer, SLIP or COBS framed, over TCP or UDP.
- ▶ **Routing**: opcodes below `0x0100` are system commands (ping, status, file transfer). Above that, the registry finds the component by fullUid and queues the command in its inbox; it answers in `handleCommand()`.
- ▶ **Every command gets an ACK or NAK** with a stage: result, queued or completed.

The action engine: observe, decide, act

Piece	What it is	In the action demo
Watchpoint	a test on a data block: component, category, offset, length, predicate, threshold	WP0: sensor temperature above 50 °C
Group	watchpoints combined with AND or OR	GP0: above 80 °C AND the overtemp flag set
Event	the ID a watchpoint or group raises	event 1
Notification	a log message tied to an event	"TEMP >80 WARNING"
Sequence	RTS or ATS steps that send commands, started by an event	RTS 1, a fault campaign
Arm control	arm or disarm watchpoints, groups and sequences at runtime	a step can switch a rule off

- ▶ **Every cycle, in order**: watchpoints, groups, events, sequences, actions. Edge-triggered, bounded loops, no allocation.
- ▶ **All of it is TPRM**: rules and sequences are data in the bank, uploaded and rescanned live. No application code decides when to react.

A sample sequence: the fault campaign

WP0 sees the temperature cross 50 °C and raises event 1, which starts RTS 1. Every step is a command to DataTransform:

Step	Command	Delay
0	SET_TARGET: entry 0 is the sensor's temperature bytes	none
1	ARM_ENTRY 0	none
2, 3	PUSH_ZERO_MASK, APPLY_ENTRY: temperature reads 0	none
4, 5	PUSH_HIGH_MASK, APPLY_ENTRY	5 cycles
6, 7	PUSH_FLIP_MASK, APPLY_ENTRY	5 cycles
8	DISARM_ENTRY 0	5 cycles
9	on completion, start RTS 2, which sends CLEAR_ALL	none

- Each step is a TOML table: target `0xCA00`, opcode `0x0603`, a timeout of 50 cycles, and what to do on timeout.

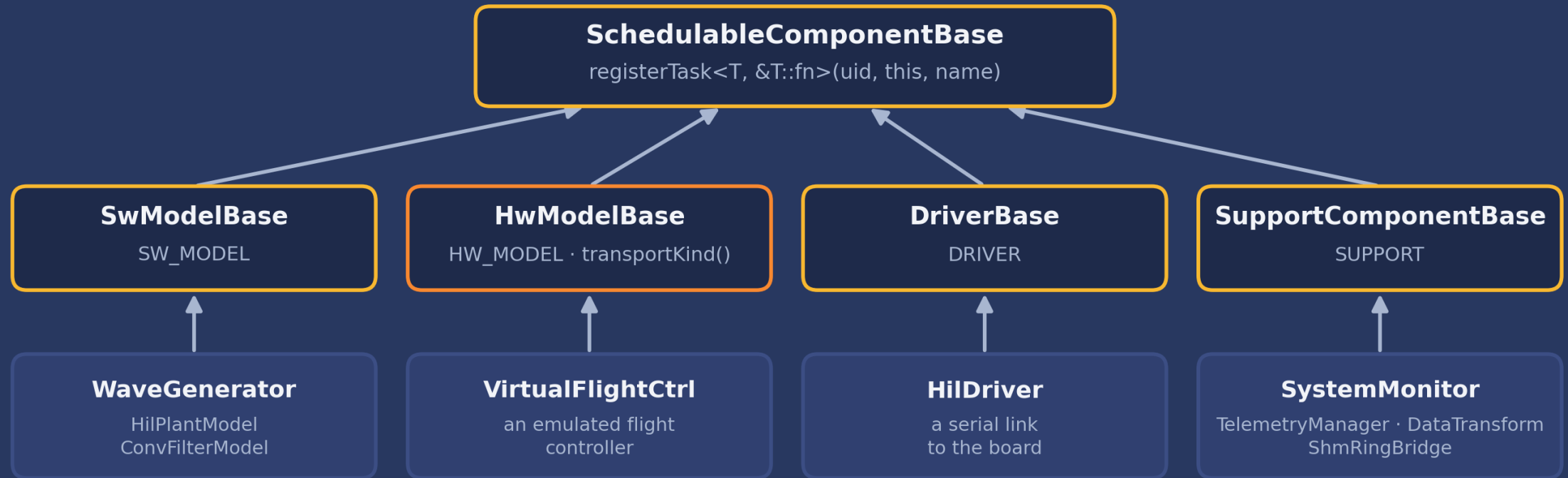
Fault injection: DataTransform

- ▶ **A byte mask on a live data block:** zero, high, flip, or a custom AND/XOR, on up to 8 targets.
- ▶ **Driven by commands** from the ground or a sequence, **or by a campaign in TPRM** that DataTransform turns into an ATS at boot:

Cycle, at 10 Hz	Fault on the sensor output
100 (10 s)	temperature zeroed, once
200 (20 s)	temperature forced high, held 5 s
350 (35 s)	temperature rate bit-flipped, once
400 (40 s)	overtemp flag zeroed, held 10 s
550 (55 s)	temperature zeroed, once

- ▶ **Detected by the same engine:** WP4, temperature equal to 0, fires "FAULT DETECTED: TEMP ZEROED". Fault, detection and response run in one loop.

Models, drivers and support



PTY pair: the same driver class talks to the model or the real board

Concrete components are final; the leaf bases mostly name the type, so the behavior lives in your class

Software to hardware in the loop, with one parameter file

- ▶ `VirtualFlightCtrl : HwModelBase` returns `TransportKind::SERIAL_232`. At registration the executive creates a PTY pair and keeps one end for the model.
- ▶ `HilDriver : DriverBase` opens whatever serial device path its TPRM file names.
- ▶ `driver_real.toml` says `/dev/ttyUSB0`: the driver talks to a NUCLEO board over USB serial.
- ▶ `driver_emulated.toml` leaves the path empty, and the executive fills in the PTY: the same driver talks to the model.
- ▶ **Same class, same code**: only a parameter file changes, and the hardware-in-the-loop rig can run both at once and compare them.

Transport kind	What the executive creates
RS-232, RS-422, RS-485	a PTY pair
SPI, I2C	a stream socket pair
CAN	a datagram socket pair

SECTION 2 · MICROCONTROLLERS

The same model, bare metal

Running example: the encryptor on four MCU families

On a microcontroller: McuExecutive and McuScheduler

	Linux	Microcontroller
Component base	SystemComponentBase	McuComponentBase: the same ComponentCore, no heap
Executive	ApexExecutive, six threads	McuExecutive<MaxTasks, Counter>, one loop and an injected tick source
Scheduler	SchedulerMultiThread, tasks from TPRM	McuScheduler<MaxTasks>, tasks added before init()

```
static executive::mcu::McuExecutive<> exec(&tickSource, 100);    // 100 Hz

exec.addTask({{profilerStartTask, &tracker}, 1, 1, 0, 127, 10});
exec.addTask({{ledBlinkTask, nullptr}, 1, 50, 0, 0, 1});        // 2 Hz
exec.addTask({{dataChannelTask, nullptr}, 1, 1, 0, 0, 2});      // 100 Hz
exec.addTask({{commandTask, nullptr}, 1, 5, 0, 0, 3});          // 20 Hz
exec.addTask({{profilerEndTask, &tracker}, 1, 1, 0, -128, 11});
```

Hardware layer: one interface, many boards

Family	IUart	ICan · ISpi · I2c	IFlash	IPps	Tick source
STM32	Stm32Uart<Rx, Tx>	Stm32Can, Stm32Spi, Stm32I2c	Stm32Flash	Stm32Pps	SysTick, timer, FreeRTOS
AVR	AvrUart<Rx, Tx>	none	AvrEeprom	AvrPps	timer
RP2040	PicoUart, PicoUsbCdc	none	PicoFlash	PicoPps	SysTick
ESP32	Esp32Uart, Esp32UsbCdc	none	Esp32NvsFlash	Esp32Pps	timer
Linux	none	none	none	LinuxPps	the executive's clock

- ▶ **Pure virtual interfaces**, `noexcept` throughout, implemented by `final` classes; a USB serial port is just another `IUart`.
- ▶ **No heap after init**: buffer sizes are template parameters, so the ring buffers live inside the object.
- ▶ **Host tests use the same headers**: define `APEX_HAL_STM32_MOCK` (or `AVR`, `PICO`, `ESP32`) and the vendor SDK drops out.

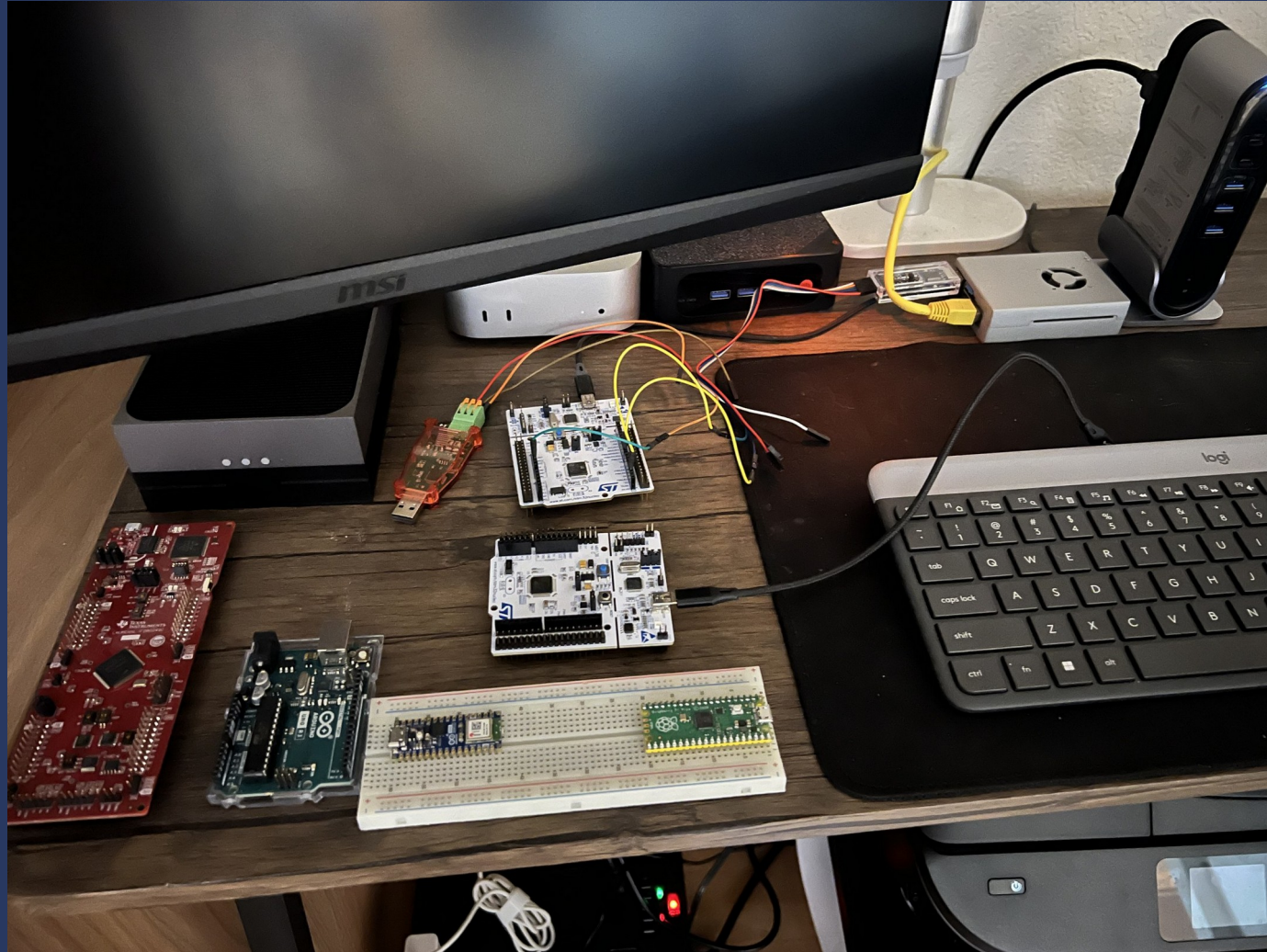
Hardware layer: sized at compile time, driven by interrupts

```
template <size_t RxBufSize = 256, size_t TxBufSize = 256> class Stm32Uart final : public IUart {
public:
    static_assert(RxBufSize > 0, "RX buffer size must be > 0");
    static_assert(RxBufSize <= 4096, "RX buffer size too large");
    ...
};

Stm32Uart<256, 128> uart(USART2, VCP);           // buffers live inside the object
extern "C" void USART2_IRQHandler() { uart.irqHandler(); }
```

- ▶ **The interrupt fills and drains** the ring buffers; `write()` copies into the TX buffer and returns at once.
- ▶ **Boards are a build option:** `-DAPEX_STM32_BOARD=nucleo_f767zi` selects one board header with its pins, clock and linker script; app code names `board::CMD_UART`, never a pin.
- ▶ **The same header builds for host tests** with `APEX_HAL_STM32 MOCK`: no vendor SDK, deterministic behavior.

Hardware in action



Demo 1: common libraries on four MCU families

The same small device on every board: it takes packets over a serial line, encrypts each one with AES-256-GCM, and sends it back.

Board	Chip	Runs on
NUCLEO L476RG · F446RE · F767ZI	STM32 Cortex-M4 and M7, 80-216 MHz	MCU executive, bare-metal or inside FreeRTOS
Arduino Uno R3	ATmega328P, 16 MHz, 2 KB of RAM	MCU executive
Raspberry Pi Pico	RP2040, dual Cortex-M0+	MCU executive
Arduino Nano ESP32	ESP32-S3, 240 MHz	MCU executive inside one FreeRTOS task

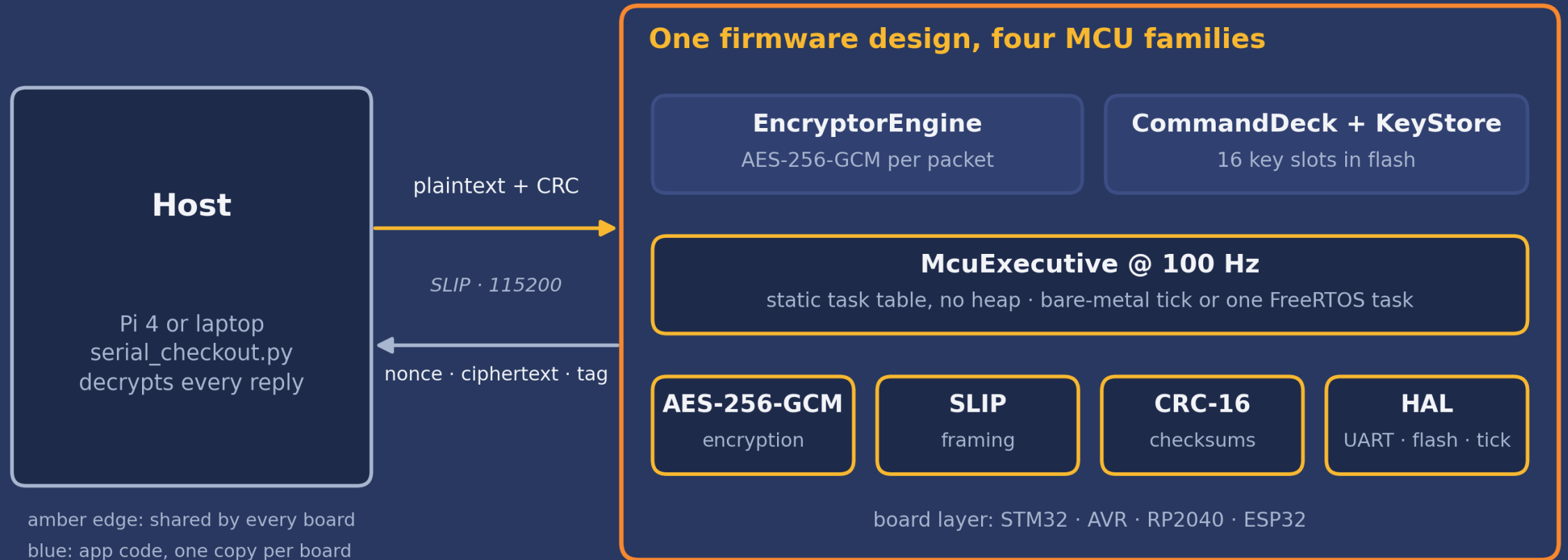
- ▶ Packets arrive SLIP-framed with a CRC-16. Each reply carries a key index, a 12-byte counter nonce, the ciphertext and a 16-byte tag.
- ▶ A command channel manages 16 key slots in flash, the nonce counter and statistics.
- ▶ A host script runs the checkout, and decrypts every reply with an independent crypto library.

Demo 1: shared libraries over a hardware layer

Layer	Shared by every board	Written per board
App	the protocol design	encryptor, command handler, key store
Executive	<code>McuExecutive</code> and <code>McuScheduler</code> : a static task table, no heap	the tick source: <code>SysTick</code> , AVR timer, Pico, ESP32 timer, FreeRTOS
Libraries	AES-256-GCM, SLIP framing, CRC-16	nothing
Hardware layer	<code>IUart</code> , <code>IFlash</code> and tick interfaces	<code>Stm32Uart</code> , <code>AvrUart</code> , <code>PicoUsbCdc</code> , <code>Esp32NvsFlash</code> , ...

- ▶ **Adding an STM32 board** is one board header of about 120 lines and a linker script, picked with one CMake option.
- ▶ **Bare-metal or FreeRTOS** from one switch: both pass 40/40 on all three NUCLEOs, and keys survive a power cycle on the F446RE and F767ZI.
- ▶ **One command per board** builds and flashes it: `make compose-stm32-flash`, `make compose-esp32-flash`.

Demo 1: one design on every board



The same host checkout runs against every board, and decrypts every reply

4

MCU families, 6 boards

40/40

checks, 3 NUCLEOs × 2 modes

20-23 KB

flash on STM32

< 0.4%

of the tick used when idle

2 KB

RAM on the smallest board

SECTION 3

CUDA in the RT loop

GPU compute inside a deterministic system, without blocking the control loop

CUDA in Apex CSF: optional acceleration, same C++

- ▶ **Kernels live in `.cu` files** behind ordinary C++ model classes, so the code that calls them never sees a CUDA type.
- ▶ **The runtime pieces used**: device memory allocated once at startup, a CUDA stream per model, completion events, and cuFFT for spectra.
- ▶ **Built with everything else**: CUDA code compiles as C++20 and cross-builds for the Jetson; about 40 .cu files live in the tree.
- ▶ **Where it runs**: the GPU compute models in Demo 2, gravity-field evaluation, batched circuit solving, and math utilities.
- ▶ **Optional by construction**: with no CUDA toolkit, the CUDA targets drop out, `COMPAT_HAVE_CUDA` is 0, and compatibility headers keep host code building.

One codebase, two very different targets

	Demo 1: microcontrollers	Demo 2: GPU
Hardware	STM32, AVR, RP2040, ESP32, down to 2 KB of RAM	Jetson AGX Thor and an RTX 5000 Ada laptop
Executive	McuExecutive: a static task table, no heap	ApexExecutive: a worker pool and runtime parameters
A task is	a function and a context: <code>addTask({{fn, &ctx}, ...})</code>	a component method: <code>registerTask<T, &T::kick></code>
Scheduled by	rate, offset and priority	rate, offset and priority, from a parameter file
Hardware access	IUart, IFlash and tick-source interfaces	CUDA streams, and NVML through Seeker
Build	<code>make compose-stm32</code>	<code>make compose-jetson-release</code>

- ▶ **Both come from one repository:** the same delegate library binds every task, the same scheduling model runs it, and the same Docker-based build produces it.
- ▶ Moving between them changes the executive and the hardware layer, not how you think about the application.

The Jetson AGX Thor: what it's for, and what it did

14 Arm Neoverse-V3AE cores · 128 GB of memory shared by CPU and GPU · Blackwell-generation GPU · CUDA 13.0

Use	What it showed
Edge compute soak	39 minutes at 100 Hz with four CUDA kernels beside the loop: every cycle ran, jitter std dev 0.36 ms
Release bundle check	a cross-built release deployed to the board reproduced all four kernels' results exactly
Scheduler re-proof	600 s under FIFO priorities after the dispatch rewrite: jitter std dev from 0.573 to 0.337 ms
Vernier CI runner	gates pull requests and runs nightly on the board; the GPU example's kernel got 10× faster under Nsight
Seeker captures	real-time checks on its cores; host-to-device copies 14× faster than a laptop's discrete GPU
Zenith targets	two ops demo instances on one board, pinned to separate cores

Demo 2: CUDA work beside a real-time loop

- ▶ **One executive on a Jetson AGX Thor runs a 10 ms frame** while four CUDA workloads run beside it: 2D convolution, batched FFT, batch statistics and stream compaction.
- ▶ **Each GPU model is two tasks:** *kick* collects the last result and launches the next kernel in microseconds; *poll* checks progress without waiting.
- ▶ **The loop never blocks on the GPU:** a kernel that takes 400 ms simply finishes on a later frame.
- ▶ **A system monitor** samples CPU, memory and GPU once a second through NVML, and flags throttling.
- ▶ **Same source on two platforms:** the Thor (aarch64) and an RTX 5000 Ada laptop (x86_64). Only the TPRM timing files differ.
- ▶ `make release APP=ApexEdgeDemo` cross-builds a bundle for the Jetson.

Demo 2: how GPU work is dispatched

- ▶ **Set up once**: at init each model allocates its GPU buffers, one CUDA stream and one completion event. Nothing is allocated on the GPU per frame.
- ▶ **kick** asks whether the last run finished. If not, it counts a busy tick and returns. If so, it collects the results, uploads new input and launches the kernels on the model's own stream.
- ▶ **poll** is a non-blocking `cudaEventQuery` that collects results as soon as they're ready.
- ▶ **Kernels** are hand-written CUDA with fixed thread-block sizes: 16×16 tiles for convolution, 256 threads per block elsewhere. The FFT uses cuFFT on the same stream.
- ▶ **Scheduled like any other task**: kick and poll are entries in the scheduler's parameter file, with a rate, an offset and a priority.

Demo 2: the kick task

```
std::uint8_t kick() noexcept {
    if (gpuInFlight_) {
        if (cudaEventQuery(completionEvent_) == cudaErrorNotReady) {
            ++s.busyCount;           // still running: skip this launch, never wait
            return 0;
        }
        harvestResults();           // finished: collect the last run
        gpuInFlight_ = false;
    }
    generateInput();
    cudaMemcpyAsync(dInput_, hInput_.data(), INPUT_BYTES, cudaMemcpyHostToDevice, stream_);
    cuda::batchStatsCuda(dInput_, p.elementCount, p.groupSize, dStats_, stream_);
    cudaEventRecord(completionEvent_, stream_);
    gpuInFlight_ = true;
    return 0;
}
```

A busy GPU costs one skipped launch. The task never waits for the GPU to finish.

Scheduling considerations for GPU work

- ▶ **Kick at normal priority, poll lower and later:** convolution kicks at tick 0 and polls at 50; FFT kicks at 12 and polls at 37, so launches don't pile onto one tick.
- ▶ **One stream, one run in flight per model:** a busy GPU turns into a counted, skipped launch, never a wait.
- ▶ **Keep CPU work off the tick:** generating input, uploading from pageable memory and a synchronous 16 MiB readback keep a worker busy across ticks.
- ▶ **Pick the real-time mode:** the stock default skips a busy tick; hard modes stop the system on a violation.
- ▶ **Priorities need the capability:** FIFO priorities need `CAP_SYS_NICE`, and per-pool affinity keeps GPU work off the control cores.
- ▶ **Next:** pinned host buffers, and moving input generation and readback off the tick path.

GPU telemetry: SystemMonitor through Seeker

- ▶ **SystemMonitor** is a support component that samples once a second. For the GPU it asks Seeker, which queries NVIDIA's NVML.
- ▶ **What it reads**: temperature, compute utilization, memory used, and the driver's throttle flags.
- ▶ **Thresholds come from TPRM**, and crossing one logs a warning or an error:

Metric	Warn	Critical
Temperature	75 °C	85 °C
Compute utilization	90%	98%
Memory used	80%	95%
Throttling	any throttle flag set	none

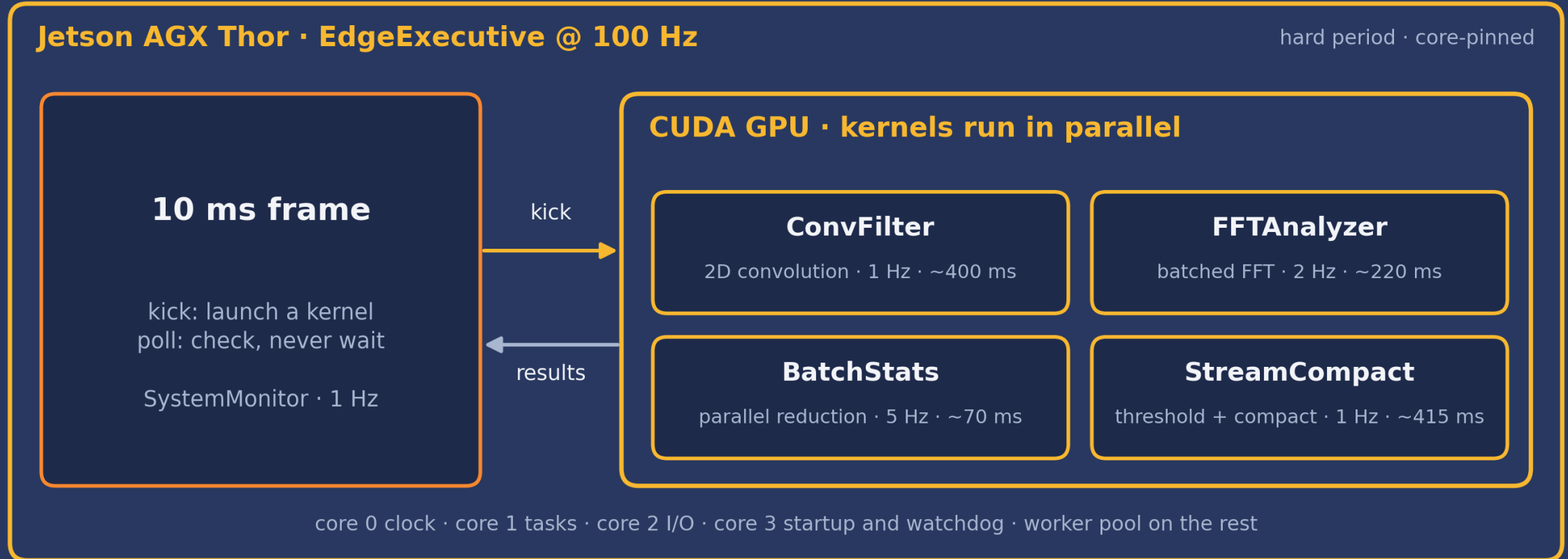
- ▶ **Off unless asked**: GPU monitoring is compiled in only when NVML is available, and enabled per app in TPRM.

Demo 2: what it proved

Result	Jetson AGX Thor	RTX 5000 Ada laptop
Soak	39 min, 233,046 cycles	39 min, 234,828 cycles
Cycle lag	0	0
GPU results	20,973	21,133
Jitter, std dev / worst	0.36 ms / 4.6 ms	0.63 ms / 6.1 ms
Kernel times, conv / FFT / stats / compact	413 / 224 / 66 / 415 ms	397 / 223 / 72 / 415 ms
GPU throttle flags	none reported	thermal and power limits set for the whole run

- **Reproducible**: a release bundle deployed to the Thor gave all four kernels' results bit for bit against the reference run.
- **Open**: these runs had no real-time priority, and a third of ticks found a worker thread still busy with the CPU side of a GPU job.

Demo 2: the layout



Same source for Jetson (aarch64) and an RTX 5000 Ada laptop (x86_64): only the TPRM timing files differ

100 Hz

executive rate

0

cycle lag, 467,874 cycles

~21K

GPU results per 39-min run

0.36 ms

jitter std dev on Thor

4

CUDA kernels in flight

SECTION 4

Validation and tooling

Benchmark the code, then check the machine it runs on

SECTION 4 · VERNIER

Measuring the code

Performance tests that are Google Test tests

What is Vernier?

A C++23 library that turns Google Test into a benchmarking framework.

- ▶ performance tests are ordinary **TESTs**, with a measurement engine underneath
- ▶ each test is timed over several **rounds**, and the result is their median, percentiles, and spread, not one number
- ▶ one flag, `--profile`, runs any of **15 profilers** on the same test binary
- ▶ **CUDA built in**: GPU code measured by the same tests, per-kernel CUPTI counters
- ▶ a companion CLI, **bench**, compares runs and fails CI on a regression

What is GoogleTest?

The C++ test framework most teams already use: github.com/google/googletest

```
TEST(Math, AddsNumbers) {  
    EXPECT_EQ(add(2, 2), 4);  
}
```

- ▶ a test is a function; tests group into suites
- ▶ `EXPECT_*` and `ASSERT_*` checks with readable failure messages
- ▶ fixtures for shared setup, parameterized tests for sweeps
- ▶ one runner: filtering, sharding, XML reports, `ctest` discovery

GoogleTest fixtures

Macro	What it's for
<code>TEST(Suite, Name)</code>	a standalone test
<code>TEST_F(Fixture, Name)</code>	a test that shares setup from a fixture class
<code>::testing::Test + SetUp() / TearDown()</code>	the fixture class: runs before and after each test
<code>SetUpTestSuite() / TearDownTestSuite()</code>	expensive setup shared by a whole suite
<code>TEST_P(Fixture, Name)</code>	one test body, run once per parameter
<code>INstantiate_Test_Suite_P</code>	the parameter values to run it with
<code>TYPED_TEST</code>	the same test across several types
<code>EXPECT_* / ASSERT_*</code>	check a value; ASSERT stops the test on failure
<code>gtest_main</code>	a ready-made <code>main()</code> that runs everything

The perf object: how vernier extends GoogleTest

A GoogleTest test checks answers. `PERF_GUARD(perf)` adds a measurement object to that same test, and its calls turn the test into a benchmark:

Call	What it does
<code>perf.warmup(fn)</code>	runs <code>fn</code> untimed before timing starts: once by default, or vernier picks a count with <code>--warmup 0</code>
<code>perf.throughputLoop(fn)</code>	calls <code>fn</code> back to back, <code>--cycles</code> times per round, on one thread. One sample per round: its time $\div$ cycles
<code>perf.contentionRun(fn)</code>	starts <code>--threads</code> threads at the same moment, each calling <code>fn</code> <code>--cycles</code> times. One sample per round: its time $\div$ (threads $\times$ cycles)
<code>perf.measured(fn)</code>	times a loop you write yourself, once per round, divided by cycles

Each call runs `--repeats` rounds and returns the **median, percentiles, mean, standard deviation, CV, calls per second**, and a **stability flag**. It also prints the result line and writes the CSV row.

The method decides the measurement: the same function timed with `throughputLoop` and with `contentionRun` gives different numbers, because they're different experiments.

Running a vernier test

A vernier test binary is an ordinary GoogleTest binary. `PERF_MAIN()` replaces `gtest_main` and adds these flags:

```
./Analyzer_PTEST --cycles 100 --repeats 5 --csv ladder.csv
./PayloadSweep_PTEST --target-time 60ms --repeats 10 --csv sweep.csv
```

Flag	What it controls	Default
--cycles N	calls per round	10,000
--repeats N	rounds per test; each round is one sample	10
--target-time 60ms	picks cycles so each round lasts about this long (us, ms, s)	off
--warmup N	untimed calls before timing; 0 lets vernier choose	1
--threads N	threads for contentionRun	1
--csv FILE	writes one row per test, for CI	off
--profile TOOL	attaches a profiler, like perf, heaptrack, or nsight	off

GoogleTest's own flags, like `--gtest_filter`, still work.

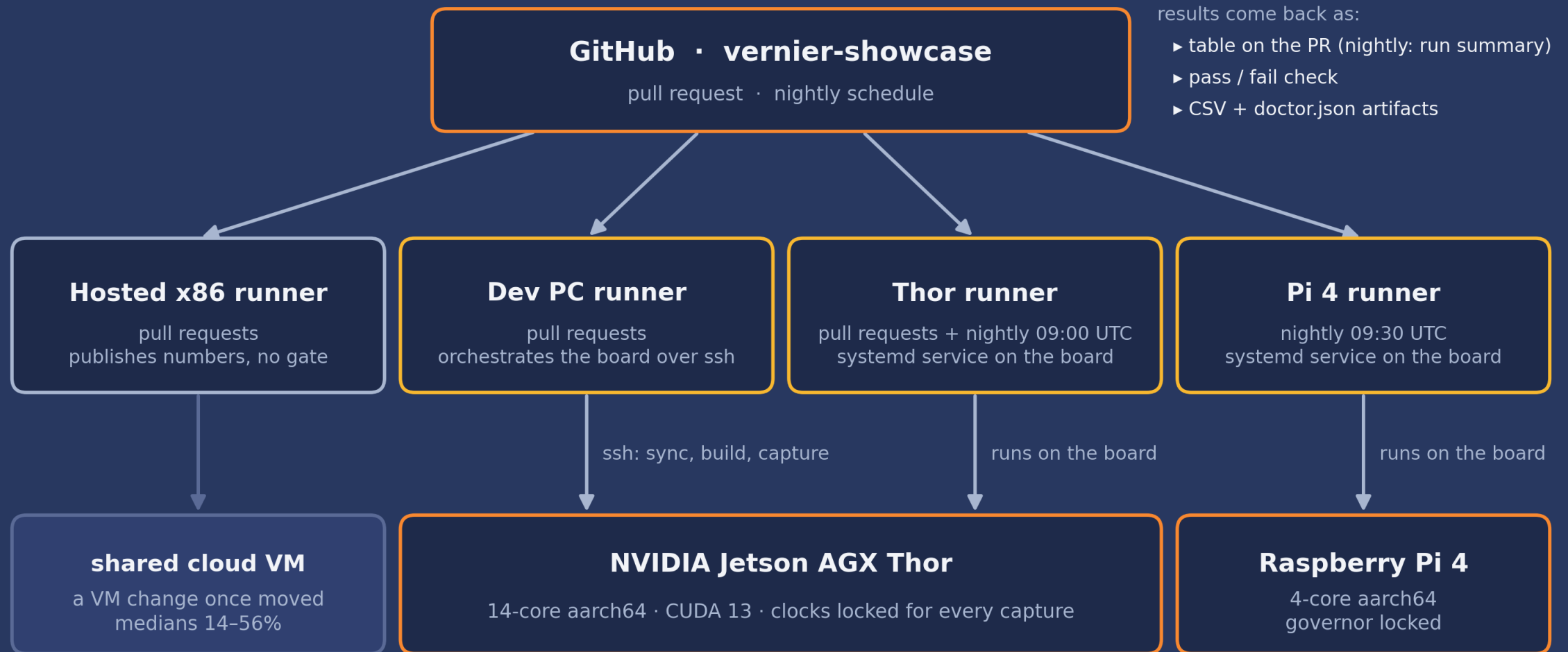
The bench CLI

The test binary measures on its own. The **bench** CLI runs it more conveniently and judges the results:

```
bench run Analyzer_PTEST --profile heaptrack -- --gtest_filter=Analyzer.V0BatchThroughput
bench compare bench/baselines/thor/baseline.csv candidate.csv --fail-on-regression
```

Command	What it does
<code>bench run BINARY [options] -- [test args]</code>	finds the binary by short name, forwards vernier's flags, and wraps profilers that must run outside the binary, like heaptrack and valgrind. <code>--taskset</code> pins it to CPUs
<code>bench compare BASELINE CANDIDATE</code>	compares two CSVs test by test. A row is a regression when it's slower by more than 5% and the difference is statistically significant
<code>bench compare ... --markdown</code>	prints the table CI posts on a pull request
<code>bench compare ... --fail-on-regression</code>	exits with an error on any regression, which fails the CI job
<code>bench doctor BINARY</code>	checks which profilers work on this machine

Example: CI on real hardware



results come back as:

- ▶ table on the PR (nightly: run summary)
- ▶ pass / fail check
- ▶ CSV + doctor.json artifacts

one board runs one job at a time · forked PRs never reach hardware

Wrapped profilers: CPU

<code>--profile</code>	Useful for
<code>perf</code>	hardware counters (cache misses, branch misses) and sampled hot spots
<code>gperf</code>	which function eats the time, as a flame graph
<code>callgrind</code>	exact instruction counts, identical every run, ideal for A/B comparisons
<code>heaptrack</code>	every allocation with its call stack, ranked by site
<code>massif</code>	heap size over time: where memory peaks, and who owns it
<code>jemalloc</code>	sampled allocation hot spots at very low overhead
<code>memcheck</code>	leaks, use-after-free, uninitialized reads
<code>helgrind</code>	data races and lock-ordering bugs in threaded code
<code>offcpu</code>	where threads block and wait, the time other profilers can't see
<code>bpfttrace</code>	kernel tracing: syscalls, scheduler events, your own probe scripts
<code>rapl</code>	CPU energy per test, on Intel

Some need root (`bpfttrace`, `offcpu`, `rapl`) or the host kernel's tools (`perf`). `bench doctor` checks each one.

Wrapped profilers: GPU

Tool	Useful for
<code>nsight</code> (Nsight Systems)	a timeline of kernels, memory copies, and CPU/GPU overlap
<code>ncu</code> (Nsight Compute)	kernel counters: memory vs compute throughput, occupancy
<code>compute-sanitizer</code>	out-of-bounds access, races, and uninitialized device memory
<code>rocprof</code>	the AMD GPU equivalent
CUPTI (built in, no flag)	per-kernel time, launch counts, and registers for tests timed with the GPU kernel harness
BENCH_NVTX_SCOPE (built in)	named ranges, so Nsight timelines show your phases

Same `--profile` flag, same test binary as the CPU tools. `ncu` needs root on Jetson, and inside a container `nsight` only prints how to run it, so profile GPUs on the host.

The example: vernier-showcase

A sample repo that uses vernier the way a team would. Its example program is written four times, V0 to V3, from the first attempt to the fastest.

Program	What it does, in plain terms
Sensor log analyzer	Reads 8,000 lines of sensor readings, like <code>imu-0,1000,2.5,0</code> . For each sensor it works out how many readings there were, their total, and the lowest and highest value.

Every version gives the same answer. Only the speed changes, and vernier measures each one with the same test.

Every number in this example was measured on one board:

Measured on	
NVIDIA Jetson AGX Thor	14-core arm64 board with an integrated GPU

CPU: the function under test

The function reads lines of text. Each line is one reading from one sensor: which sensor, when, the value it measured, and a status flag.

For each sensor, it counts the readings and tracks their total, lowest, and highest value. Here are three lines and what it returns for them:

```
imu-0,1000,2.5,0
imu-0,1250,-1.5,1
baro-0,1500,100.0,0
```

Output field	For the three records above
perSensor["imu-0"]	count 2 · sum 1.0 · min −1.5 · max 2.5
perSensor["baro-0"]	count 1 · sum 100.0 · min 100.0 · max 100.0
droppedLines	0 — broken lines, like a cut-off line or a value that isn't a number, are counted here
textReport	a printable summary, one line per sensor: imu-0: n=2 mean=0.500000 min=-1.500000 max=2.500000

CPU: the performance test, line by line

```
PERF_THROUGHPUT(Analyzer, V0BatchThroughput) { // 1
  PERF_GUARD(perf); // 2
  const auto LINES = showcase::generateRecords(BATCH_LINES, SEED); // 3
  perf.warmup([&] { showcase::analyzeV0(LINES); }); // 4
  const auto RESULT =
    perf.throughputLoop([&] { showcase::analyzeV0(LINES); }, "analyze_v0"); // 5
  EXPECT_GT(RESULT.callsPerSecond, 5.0); // 6
}
```

#	What it does
1	a GoogleTest TEST ; the name says it measures throughput
2	creates perf , the measurement object, and attaches a profiler if --profile is set
3	builds the same 8,000 records every run (fixed seed), outside the timing
4	runs the call untimed first (once, by default), so caches and the allocator are warm
5	times --cycles calls, --repeats times over; each round's average is one sample
6	a loose floor for catastrophes; CI compares the median against a baseline

CPU: reading the result line

Every vernier test prints one line like this, and writes the same numbers to a CSV row:

```
[Analyzer.V0BatchThroughput] 1365.670 us/call CV=0.1% ~732 calls/s (p10=1363.860 p90=1367.130 sd=1.506)
```

Piece	What it means
1365.670 us/call	the median of the 5 round averages: the time for one call
CV=0.1%	standard deviation ÷ mean, as a percentage: how much the rounds disagreed. Here, almost not at all
~732 calls/s	the same median, as a rate
p10 / p90	the 10th and 90th percentile across the rounds
sd	the standard deviation of the rounds, in microseconds

Thor CI run, 2026-09-12: `--cycles 100 --repeats 5`, so 500 timed calls in 5 rounds. These numbers describe the **rounds**, not individual calls.

CPU: V0, how it handles one line

```
for (const std::string& line : lines) {  
    const std::vector<std::string> FIELDS = splitFields(line);  
    if (FIELDS.size() != 4) { ++report.droppedLines; continue; }  
    double value = std::stod(FIELDS[2]);  
    SensorSummary& totals = report.perSensor[FIELDS[0]];  
    ++totals.count; totals.sum += value;  
}  
// "imu-0,1000,2.5,0"  
// 1. copy each piece into a new list  
// 2. not four pieces? drop it  
// 3. turn "2.5" into 2.5  
// 4. find imu-0's totals by name  
//    and update them
```

- ▶ Every line gets a **new list** holding a **new copy** of each piece: "imu-0", "1000", "2.5", "0". That list needs heap memory, and asking the allocator is slow next to reading a few characters
- ▶ `std::stod` takes a `std::string`, which is part of why V0 copies the pieces out. It also throws on bad input, so V0 wraps it in `try/catch`
- ▶ `perSensor[FIELDS[0]]` finds the sensor's totals using its name as a `std::string`
- ▶ Correct and readable: **1,365.7 μ s** per 8,000 lines on the Thor

CPU: ask a tool where the time goes

heaptrack counts every time a program asks for new memory. Vernier runs it with one flag, on the same test:

```
bench run Analyzer_PTEST --profile heaptrack -- \
  --gtest_filter=Analyzer.V0BatchThroughput --cycles 11 --repeats 1 --warmup 0
```

Run	Memory allocations
1 batch of 8,000 lines	151,751
11 batches	390,791
Each extra batch	23,904 , about 3 for every line

Asking for memory is slow next to reading a line, and V0 does it about three times per line: step 1's new list and copies.

CPU: V1, views instead of copies

V0

copies each piece



V1

points into the line



a view = where a piece starts + its length
no copies, no new memory

- ▶ V0 **copies** each piece of the line into new memory, on every line
- ▶ V1 leaves the line where it is and records **where each piece starts and how long it is**. That pair is a `std::string_view`
- ▶ No new memory for the pieces means the per-line allocations disappear

CPU: V1, the code

```
for (const std::string& line : lines) {                                // "imu-0,1000,2.5,0"
    std::string_view fields[4];                                       // 1. four views, on the stack
    if (!splitFieldsInPlace(line, fields)) { ++report.droppedLines; continue; } // 2.
    double value = 0.0;
    std::from_chars(fields[2].data(), fields[2].data() + fields[2].size(), value); // 3. read 2.5 in place
    auto it = report.perSensor.find(fields[0]);                       // 4. look up imu-0 with the view
    if (it == report.perSensor.end())
        it = report.perSensor.emplace(std::string(fields[0]), SensorSummary{}).first; // new sensor: copy name once
    ++it->second.count; it->second.sum += value;
}
```

- ▶ `std::string_view fields[4]` is an array on the stack, so making the four views needs **no heap memory**
- ▶ `std::from_chars` parses the characters where they sit: **no copy, no exception, no locale**
- ▶ `find()` takes the view because the map is declared with `std::less<>`. `emplace` copies the name only for a **new** sensor

CPU: V0 → V1, the result

	V0	V1	
Time per 8,000 lines, Thor	1,365.7 μs	626.1 μs	2.2× faster
Memory allocations per batch	23,904	15	about 1,600× fewer

Same answers, same unit tests. V1 just stopped copying every line apart.

The loop in one example: **vernier** measured it, **heaptrack** explained it, and the fix changed four lines.

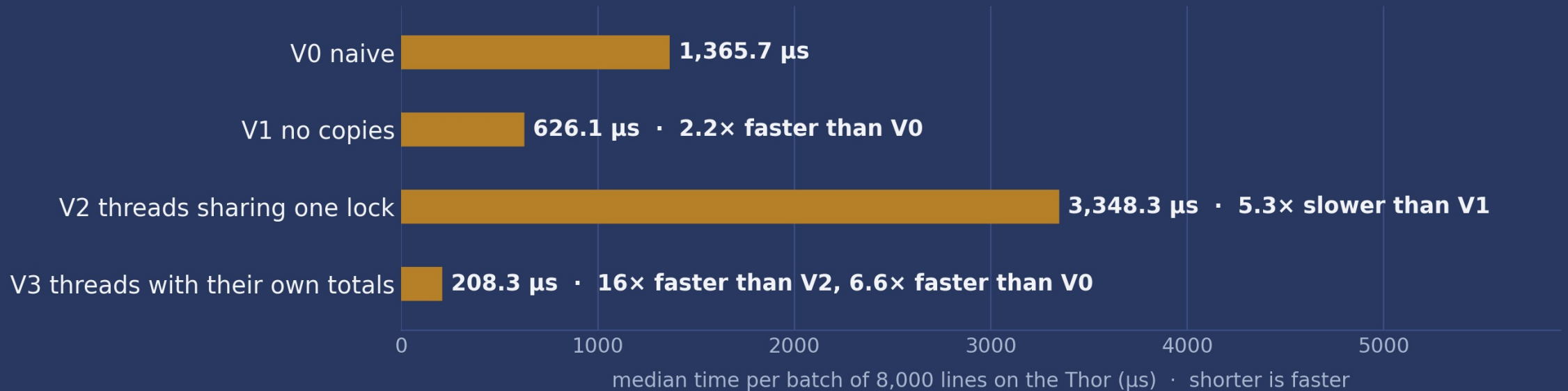
CPU: V2 and V3, in brief

Both versions use V1's copy-free parsing and split the 8,000 lines across **4 threads**. They differ in how the threads store their totals.

	How the threads keep totals	Thor	What happened
V2	All 4 threads update one shared set of totals , protected by a lock. Only one thread can hold the lock at a time.	3,348.3 μ s	5.3$\times$ slower than V1. Every line needs the lock, so the threads mostly wait for each other. An off-CPU profile showed them repeatedly stopping.
V3	Each thread keeps its own totals while it works. When all 4 finish, their totals are added together once.	208.3 μs	6.6$\times$ faster than V0. No thread ever waits for another.

The lesson from V2: adding threads can make code slower. You only find that out by measuring.

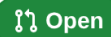
CPU: the whole ladder



- ▶ **V0 → V1:** stop copying every line apart. 2.2× faster on the Thor. *Found with heaptrack.*
- ▶ **V1 → V2:** add threads that share one set of totals. 5.3× *slower*. *Found with off-CPU profiling.*
- ▶ **V2 → V3:** give each thread its own totals. 16× faster than V2, 6.6× faster than V0. *Confirmed by the same test.*

CI: an innocent pull request

Refactor V1 field splitting for lifetime safety #4

 **Open** [kalex-g](#) wants to merge 1 commit into [dev/skeleton](#) from [dev/v1-splitter-refactor](#) 

 Conversation **3**  Commits **1**  Checks **3**  Files changed **1**



kalex-g commented [last week](#)

Contributor ...

Small cleanup while reading the parser: the splitter's raw-array out-parameter and view-typed fields made lifetime reasoning harder than it needs to be — views into the line buffer are only valid while the caller keeps the line alive. This returns owned fields in a standard container instead, so the split result is self-contained and the helper reads as ordinary modern C++.

No behavior change: malformed-line handling (fewer than four fields, or five-plus) is identical, and the full unit suite passes (16/16).

```
// before: views into the line, no allocation
bool splitFieldsInPlace(std::string_view line, std::string_view (&fields)[4]);
// after: owned strings in a standard container
std::vector<std::string> splitFields(std::string_view line) {
    std::vector<std::string> fields;
    fields.reserve(4);                                // a heap allocation for every line
```

Reads like a safe cleanup, and every unit test passes. It puts an allocation back into V1's per-line work.

CI: the report on the pull request

github-actions Bot commented [last week](#)

Benchmark comparison (thor)

Test	Baseline	Candidate	Delta	%	Result
Analyzer.V0BatchThroughput	1364.94000	1361.80000	-3.14000	-0.2%	[] neutral
Analyzer.V0SingleLineLatency	0.80000	0.79000	-0.01000	-1.3%	[] neutral
Analyzer.V1BatchThroughput	625.79000	795.10000	+169.31000	+27.1%	[!] REGRESSION
Analyzer.V1SingleLineLatency	0.67000	0.69000	+0.02000	+3.0%	[] neutral
Analyzer.V3BatchThroughput	210.02000	207.03000	-2.99000	-1.4%	[] neutral
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_2	1.65750	1.67250	+0.01500	+0.9%	[] neutral
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_2048	148.36100	202.80500	+54.44400	+36.7%	[!] REGRESSION
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_32	11.65620	12.11000	+0.45380	+3.9%	[] neutral
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_8	5.86625	6.01250	+0.14625	+2.5%	[] neutral

Both flagged rows run V1's code: the **V1 batch (+27.1%)** and the **2,048-line sweep (+36.7%)**. The runner on the board flagged the same two; V3 stayed **neutral**.

CI: the build goes red

The screenshot displays a GitHub Actions workflow run for the `dev/v1-splitter-refactor` branch. The workflow is triggered by a pull request from user `kalex-g`. The status is **Failure**, with a total duration of **27s** and **1** artifact. The workflow file is `bench-thor.yml`, which is triggered on `pull_request`. A job named `Benchmark gate (thor)` has failed, indicated by a red 'x' icon and a duration of 24s. The annotations section shows 1 error and 1 warning, with the error message: `Benchmark gate (thor)` Process completed with exit code 1.

Triggered via pull request last week	Status	Total duration	Artifacts
kalex-g opened #4 <code>dev/v1-splitter-refactor</code>	Failure	27s	1

bench-thor.yml
on: pull_request

Benchmark gate (thor) 24s

Annotations
1 error and 1 warning

Benchmark gate (thor)
Process completed with exit code 1.

The gate compares against the baseline and **exits with an error on any regression**, so the check fails on the pull request.

The unit tests still passed (16/16, run on the x86 lane): the output didn't change, only the speed. The benchmark gate on the Thor is what caught it.

SECTION 4 · SEEKER

Checking the machine

Is this machine ready for real-time work?

Check the machine before the code

Correct, fast code still misses deadlines on a machine set up for something else:

- ▶ **interrupts** land on the core running your control loop
- ▶ the **CPU governor** saves power by slowing down, and speeds up too late
- ▶ **deep sleep states** take hundreds of microseconds to wake from
- ▶ memory gets **swapped to disk** and has to come back first

None of these are in your code. All of them can be checked before you ship to a robot.

Seeker is the tool that checks them.

What is Seeker?

A C++ library and 34 command-line tools that answer one question: **is this Linux machine ready for real-time work?**

Area	What it checks
CPU	core layout, sleep states, speed governor, interrupts, reserved cores
Memory	hugepages, memory locking, swap, NUMA, ECC errors
Storage	I/O scheduler, read-ahead, fsync latency
Network	card interrupts, packet batching, socket buffers
Timing	clock source, timer slack, tick-less cores, sleep jitter
System	real-time kernel, permissions, limits, containers
GPU	persistence mode, compute mode, throttling, PCIe link

Every check prints **PASS**, **WARN**, **FAIL** or **SKIP**, plus the fix, and `--json` for CI. **Today: CPU and GPU.**

How Seeker works

TOOLS	cpu-rtcheck	mem-info	gpu-stat	...	judge: PASS / WARN / FAIL
DOMAINS	cpu	memory	storage	network	report: facts in plain structs (no domain depends on another)
	timing	device	system	gpu	
HELPERS	small file reads, string parsing, formatting				
LINUX	/proc	/sys	ioctl	NVML	missing? zeroed fields, keep going

- ▶ **One question, one function call:** `auto topo = seeker::cpu::getCpuTopology();`
- ▶ **Layers only look down:** a tool can change without touching the library
- ▶ Link only what you use: `seeker::cpu`, `seeker::gpu`, ...

The paradigms we follow

Paradigm	In Seeker	Why it helps
Data-oriented design	plain structs with fixed-size arrays, no class families	shaped like the data Linux gives you; cheap to copy, store and compare
Functional core, imperative shell	pure math in the middle, file reads and printing at the edges	the math has no side effects, so it tests without hardware
Report, then judge	the library returns facts; the tool decides PASS or FAIL	the same facts feed a dashboard, CI, or a startup check
Fail soft	missing file or failed call gives zeroed fields, never a crash	one binary runs on a laptop, a Jetson or a Pi
Choose at compile time	<code>__has_include(<nvml.h>)</code> picks the GPU code path	no runtime cost, no class hierarchy to maintain

Two machines, two different jobs

	Laptop	Jetson AGX Thor
CPU	Intel Core Ultra 7 165H: 16 cores, 22 threads, 3 core types	14 ARM Neoverse-V3AE cores
Memory	62 GB	128 GB, shared by CPU and GPU
GPU	NVIDIA RTX 5000 Ada, 16 GB, on PCIe	NVIDIA, built into the chip
Software	Ubuntu 22.04, Linux 6.8	Ubuntu 24.04, Linux 6.8 (Jetson)
Real-time kernel	no	no

A laptop tuned for battery life, and a robot computer running stock settings.

cpu-info: how many cores, and which kind?

```
laptop — cpu-info
$ cpu-info

=== Topology ===
Packages:      1
Physical cores: 16
Logical CPUs:   22
Threads/core:  1-2 (mixed core types)
NUMA nodes:    1
L3 cache:      24.0 MiB (64-byte line, 12-way)
```

```
thor — cpu-info
$ cpu-info

=== Topology ===
Packages:      1
Physical cores: 14
Logical CPUs:   14
Threads/core:   1
NUMA nodes:    1
```

- ▶ **Cores 16 vs 14:** the laptop shows 22 CPUs because 6 cores run 2 threads → budget real cores, not CPUs
- ▶ **Threads/core 1-2 vs 1:** the laptop mixes 3 core types; Thor's cores are identical → run the loop on a whole fast core
- ▶ **NUMA nodes 1:** all memory equally close → on big servers, keep a thread and its memory together
- ▶ **L3 cache 24 MiB:** shared by every laptop core → keep busy neighbors off the machine

cpu-info: what can this CPU do?

```
laptop — cpu-info
$ cpu-info

=== CPU Features ===
Vendor: GenuineIntel
Brand: Intel(R) Core(TM) Ultra 7 165H
SIMD: SSE SSE2 SSE3 SSSE3 SSE4.1 SSE4.2 AVX AVX2 FMA
Crypto: AES SHA
Other: POPCNT BMI1 BMI2 RDRAND RDSEED
Invariant TSC: yes
```

```
thor — cpu-info
$ cpu-info

=== CPU Features ===
Vendor: ARM
Brand: ARM Neoverse-V3AE
MIDR: implementer 0x41, part 0xd83
SIMD: NEON SVE SVE2 DOTPROD I8MM BF16
Crypto: AES PMULL SHA1 SHA2 SHA3 SHA512 CRC32
Other: LSE
```

- ▶ **Vendor / Brand:** Intel Core Ultra 7 vs ARM Neoverse-V3AE → decides which tuning and timers apply
- ▶ **SIMD:** AVX2 on the laptop, NEON / SVE / SVE2 on Thor → compile for exactly this CPU
- ▶ **Crypto:** AES and SHA in silicon → encrypted telemetry stays cheap inside the loop
- ▶ **Invariant TSC: yes:** the laptop clock keeps a steady rate → trustworthy timestamps (Thor uses ARM's timer)

cpu-info: how fast is it running right now?

```
laptop — cpu-info
$ cpu-info

=== Frequency ===
Governor: powersave (all cores)
Current:  400 - 4406 MHz
Range:    400 - 5000 MHz (max varies by core type)
Turbo:    available
```

```
thor — cpu-info
$ cpu-info

=== Frequency ===
Governor: schedutil (all cores)
Current:  972 - 1728 MHz
Range:    972 - 2601 MHz
```

- ▶ **Governor powersave / schedutil:** speed rises only after work arrives → performance on control cores
- ▶ **Current 400-4406 / 972-1728 MHz:** idle cores sit near the bottom → control cores stay at max
- ▶ **Range up to 5000 MHz, varies by core type:** → pin to the fastest cores, set min = max
- ▶ **Turbo available:** bursts until the chip heats up → turn it off for steady speed

cpu-rtcheck: are cores 12 and 13 ready for a control loop?

```
thor — cpu-rtcheck --cpus 12-13
$ cpu-rtcheck --cpus 12-13

Target CPUs: {12,13}

[WARN] CPU Isolation      Incomplete isolation: missing isolcpus: {12,13}; missing nohz_full: {12,13}; missing rcu_nocbs: {12,13}
      -> Ensure all RT CPUs have isolcpus, nohz_full, and rcu_nocbs
[WARN] CPU Governor       2 CPU(s) not using 'performance': [12,13]
      -> Set governor: cpupower frequency-set -g performance
[WARN] C-State Latency    2 high-latency C-state(s) enabled: cpu12/cc7 (10000us), cpu13/cc7 (10000us)
      -> Disable deep idle states: cpuidle.off=1, or write 1 to cpuidle/stateN/disable
```

- ▶ **Target CPUs {12,13}**: the cores the control loop will own (--cpus 12-13)
- ▶ **Isolation WARN**: Linux still runs other work there → boot with `isolcpus`, `nohz_full`, `rcu_nocbs`
- ▶ **Governor WARN**: both cores ramp speed on demand → `performance`
- ▶ **C-State WARN**: deepest sleep takes **10 ms** to wake → disable it on 12-13

cpu-rtcheck: is anything interrupting those cores?

```
thor — cpu-rtcheck --cpus 12-13
$ cpu-rtcheck --cpus 12-13

[PASS] IRQ Affinity      No device IRQs on RT cores during 101 ms sample
[PASS] Softirq Load     RT core softirq rate < 1000/s (max: 208/s on cpu12)
[SKIP] Invariant TSC     Not applicable (x86-only; this CPU uses its architected timer)

Summary: 2 passed, 3 warnings, 0 failed

Verdict: PARTIALLY RT-READY (review warnings)
```

- ▶ **IRQ Affinity PASS:** no disk or network interrupts hit 12-13 in the sample → pin them away, re-check under load
- ▶ **Softirq 208/s PASS:** deferred kernel work is light → `nohz_full` keeps it that way
- ▶ **Invariant TSC SKIP:** x86-only check on an ARM board
- ▶ **Verdict:** 2 PASS, 3 WARN, exit code 1 → fix, re-run, ship only on exit code 0

gpu-info: a GPU card vs a GPU inside the chip

```
laptop — gpu-info
$ gpu-info

=== GPU 0 ===
Name:      NVIDIA RTX 5000 Ada Generation Laptop GPU
Vendor:    NVIDIA
UUID:      GPU-xxxxxxxx
Compute:   SM 8.9
Memory:    16.0 GiB
PCIe:      0000:01:00.0 (x8 Gen4)
           -> Max: x16 Gen4
Driver:    580.178.04 (CUDA 13.1)
Compute mode: Default
Persistence: disabled
```

```
thor — gpu-info
$ gpu-info

=== GPU 0 ===
Name:      NVIDIA Thor
Vendor:    NVIDIA
UUID:      GPU-xxxxxxxx
Compute:   SM 11.0
Memory:    not reported
PCIe:      0000:01:00.0 (link info not available)
Driver:    580.00 (CUDA 13.0)
Compute mode: Default
Persistence: enabled
```

- ▶ **Compute SM 8.9 vs 11.0:** GPU generation → build CUDA code for exactly this one
- ▶ **Memory 16 GiB vs not reported:** the card has its own; Thor shares 128 GB with the CPU → allocate once at startup
- ▶ **PCIe x8 of x16 vs none:** data crosses a bus only on the laptop → measure copy time
- ▶ **Persistence off vs on:** the laptop driver can unload when idle → turn persistence on

gpu-rtcheck: is the laptop GPU ready for real-time work?

```
laptop — gpu-rtcheck
$ gpu-rtcheck

GPU 0: NVIDIA RTX 5000 Ada Generation Laptop GPU
-----
[WARN] Persistence Mode      Persistence mode disabled
      -> Enable with: nvidia-smi -pm 1
[WARN] Compute Mode          Default mode (shared access)
      -> Set exclusive mode: nvidia-smi -c EXCLUSIVE_PROCESS
[PASS] Temperature           44 C (good)
[FAIL] Throttling             Thermal throttling active
      -> Improve cooling or reduce GPU load
[SKIP] ECC Memory             ECC not supported by hardware
[SKIP] Retired Pages           ECC not supported (no page retirement tracking)
[PASS] Driver Versions        Driver 580.178.04 (supports CUDA 13.1)
[WARN] PCIe Link              Running at x8 Gen4 (max: x16 Gen4)
      -> Check slot seating and motherboard slot capability
[WARN] Process Isolation      1 process(es) active (0 compute, 1 graphics)
      -> Consider exclusive mode or dedicated GPU for RT workloads

Result: 2 passed, 4 warnings, 1 failed
Verdict: NOT RT-READY
```

- ▶ **Persistence, Compute Mode WARN:** driver unloads, desktop shares the GPU → `nvidia-smi -pm 1`, exclusive mode
- ▶ **Throttling FAIL at 44 C:** the driver already caps power and heat → pick a GPU with headroom
- ▶ **Driver PASS:** supports CUDA 13.1
- ▶ **PCIe WARN:** x8 of x16 → on a desktop, move the card
- ▶ **Process Isolation WARN:** the desktop uses it → dedicate a GPU to control work

gpu-bench: a GPU card vs a GPU built into the chip

Same tool, 64 MB copies with pinned memory, `gpu-bench --quick`:

	Laptop: RTX 5000 Ada on PCIe	Jetson AGX Thor: GPU in the chip
Copy to GPU	7,976 MiB/s (8.0 ms per copy)	111,143 MiB/s (0.58 ms per copy)
Copy back from GPU	7,674 MiB/s	111,002 MiB/s
Start one GPU kernel	1.82 μ s	0.81 μ s
Allocate GPU memory	1.4 ms	0.7 ms

- ▶ **Checks:** what moving data and launching work costs on *this* machine
- ▶ **Here:** on the laptop every frame pays a trip across the PCIe bus; on Thor CPU and GPU share memory
- ▶ **So:** measure the copy, not just the kernel, before choosing hardware for a control loop

Use it in CI and in your code

```
# CI: first step on every board
cpu-rtcheck --json > cpu-rtcheck.json
echo $?          # 0 = pass    1 = warnings    2 = failures
```

```
// find_package(seeker) -> target_link_libraries(app PRIVATE seeker::cpu)
#include "src/cpu/inc/CpuTopology.hpp"

// at startup (NOT RT-safe): read once, keep the answer
static const auto TOP0 = seeker::cpu::getCpuTopology();
```

- ▶ **Check every board, every boot:** a setting that changed after an update is caught before it causes a missed deadline
- ▶ Read static facts once at startup; call only RT-safe functions inside the loop

SECTION 5

Simulation architecture

Vehicles composed from reusable physics libraries

Simulation libraries: physics you compose

Area	What it provides	Examples
Dynamics	state integration and rigid-body motion	RK4, a 6DOF rigid body with quaternion attitude, mass properties, Dryden turbulence
Aerodynamics	forces and moments in body axes	polar model, stability-derivative model
Propulsion	thrust from throttle and air density	density-scaled thrust, a two-spool turbofan
GNC	control loops	a PID loop; pitch, altitude, speed, heading and roll holds; a yaw damper
Sensors	truth turned into readings, with seeded noise and bias	GPS, pitot, radar altimeter, box-clearance lidar
Environment	the planet	atmosphere tables, gravity models, terrain, a celestial body

- **Libraries hold the physics; components hold the schedule.** The libraries are header-only, and no vehicle's numbers are baked in.

Composing the aircraft

```
ApexExecutive, 100 Hz
├─ CelestialBody      SW_MODEL  atmosphere table, gravity, terrain
├─ Aircraft           SW_MODEL  100 Hz: 6DOF rigid body, aero, turbofans,
                             fuel burn, Dryden gusts, pitot
├─ AircraftController SW_MODEL  25 Hz: six loops built on PIDLoop
├─ Action engine      CORE      flight-mode set-pieces, envelope watchpoints
└─ ShmRingBridge      SUPPORT   256-byte frame to Unreal, commands back
```

- ▶ **Wired by pointer at startup:** the aircraft reads its celestial body, the controller reads the aircraft.
- ▶ **Every physical model is a library object inside a component;** the components only decide when it runs.

Inside one aircraft tick

1. Read the air density from the celestial body.
 2. Step the Dryden gust filters; the pitot model turns airspeed into a reading.
 3. Step each turbofan's spools and add up the thrust.
 4. Burn fuel and update the mass properties.
 5. Evaluate the stability-derivative aerodynamics at every RK4 stage.
 6. Advance the 13-state rigid body, with quaternion attitude.
- The autopilot runs every fourth tick, at 25 Hz; the frame goes to Unreal every tick.

Reuse across demos

Shared piece	Used by
CelestialBody	the aircraft, the rover on terrain (Earth and Moon), the world query probe
Sensor models	the aircraft's pitot, the drone lidar box's six lidar pods
Frame and quaternion math	every vehicle, and checked on an STM32 against the host's answers
World data files	Apex CSF and Unreal load the same terrain and atmosphere, and compare every frame

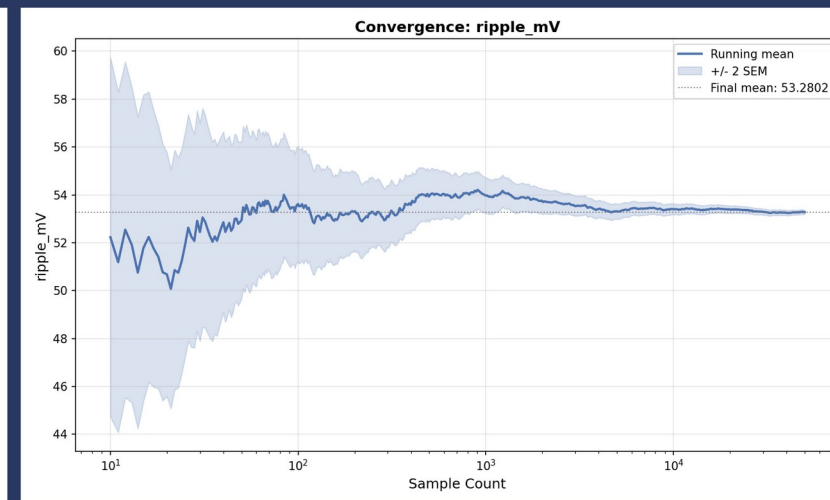
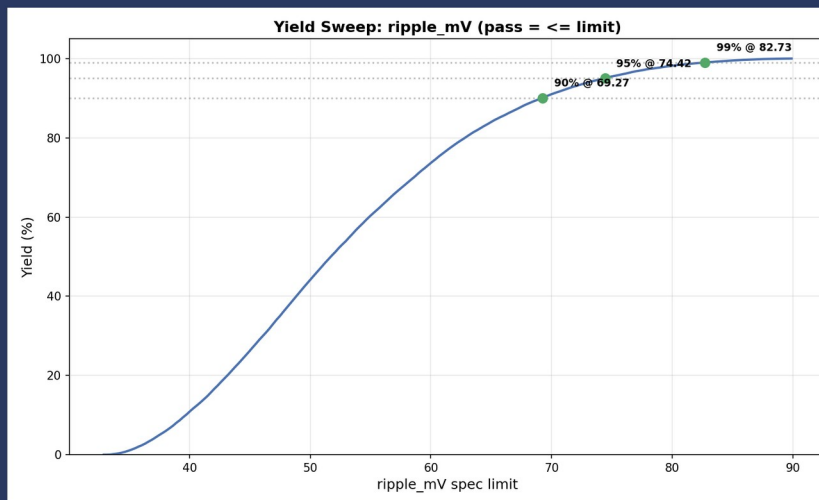
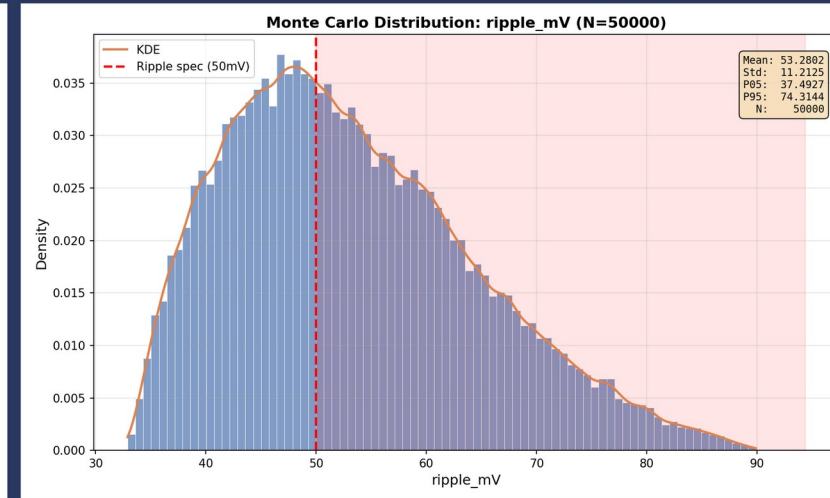
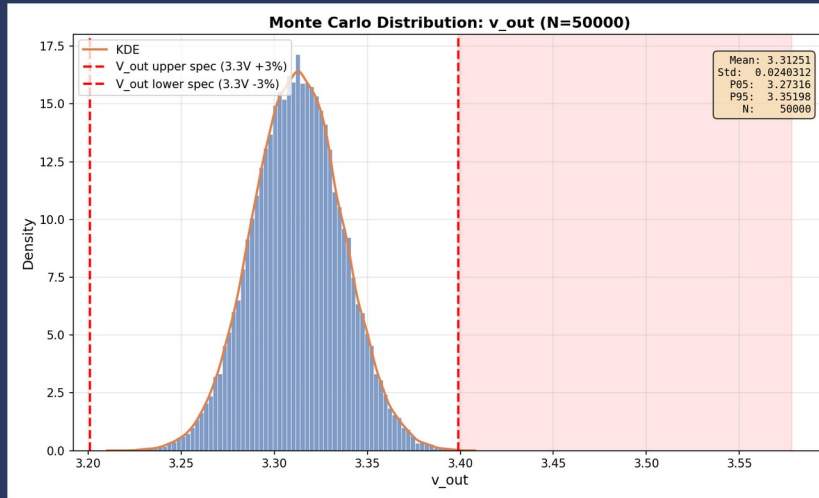
- The dynamics and control libraries are written for any vehicle; the aircraft is their first full user.

Monte Carlo: thousands of boards on every core

- ▶ **A header-only driver:** an atomic counter hands run indices to a lock-free thread pool, and results land in pre-allocated slots, so workers never contend.
- ▶ **Each run is a function call**, and can be a whole simulation: it can even run an `McuExecutive` in fast-forward.
- ▶ **Sweeps:** grid, uniform random, Latin hypercube, Cartesian product, or your own mutator. Each run is seeded from the base seed plus its index, so results don't depend on thread count.
- ▶ **Out:** mean, standard deviation, percentiles, yield against spec limits, a convergence check, and CSV for plotting.

The demo: a 3.3 V regulator, 50,000 boards	Mean	Yield
Output voltage , spec $\pm 3\%$	3.313 V	99.99%
Ripple , spec 50 mV or less	53.3 mV	44.2%
Phase margin , spec 45° or more	93.9°	100%
All specs		44.2%

Monte Carlo: reading the plots



What else can you do?

Use	What the demo shows
Simulate down to the transistor	an Intel 4004 running real programs at three fidelity levels, from instruction-level to transistor physics, picked with one flag
Distribute time	PPS ingest, UTC correlation, time-triggered sequences, and recovery when the GPS signal drops out
Generate components from specs	specification files become component code, and CI checks the generated code hasn't drifted
Monitor health and push telemetry	CPU, memory and GPU thresholds, and subscribed data blocks pushed to the console without polling

SECTION 6

Visualization & operations

Unreal Engine over shared memory, and a web console for every target

SECTION 6 · HORIZON

Seeing it

Live 3D views of a running simulation

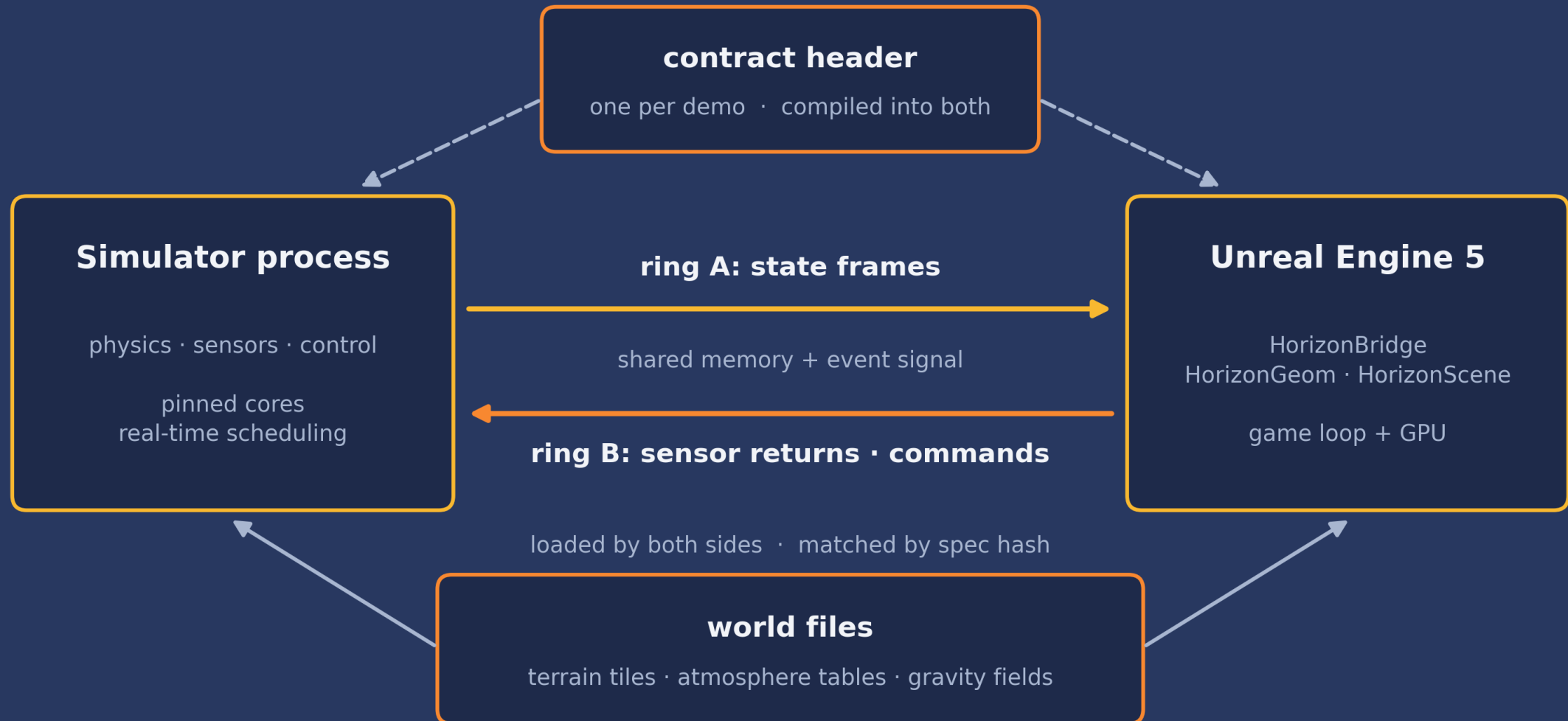
What is Horizon?

Connects a running simulation to Unreal Engine 5 for a live 3D view.

- ▶ **separate processes**: the simulator keeps its real-time loop, Unreal its game loop
- ▶ **shared-memory rings** in both directions: state frames out, sensor returns and commands back
- ▶ **one contract header** per demo, compiled into both sides and checked when they connect
- ▶ **scenes built in code**: lighting, terrain, beams, cameras; NASA or generated vehicle models
- ▶ **shared world files**: terrain, atmosphere and gravity, loaded by both, matched by hash
- ▶ C++20, Unreal Engine 5.7 plugins, Linux

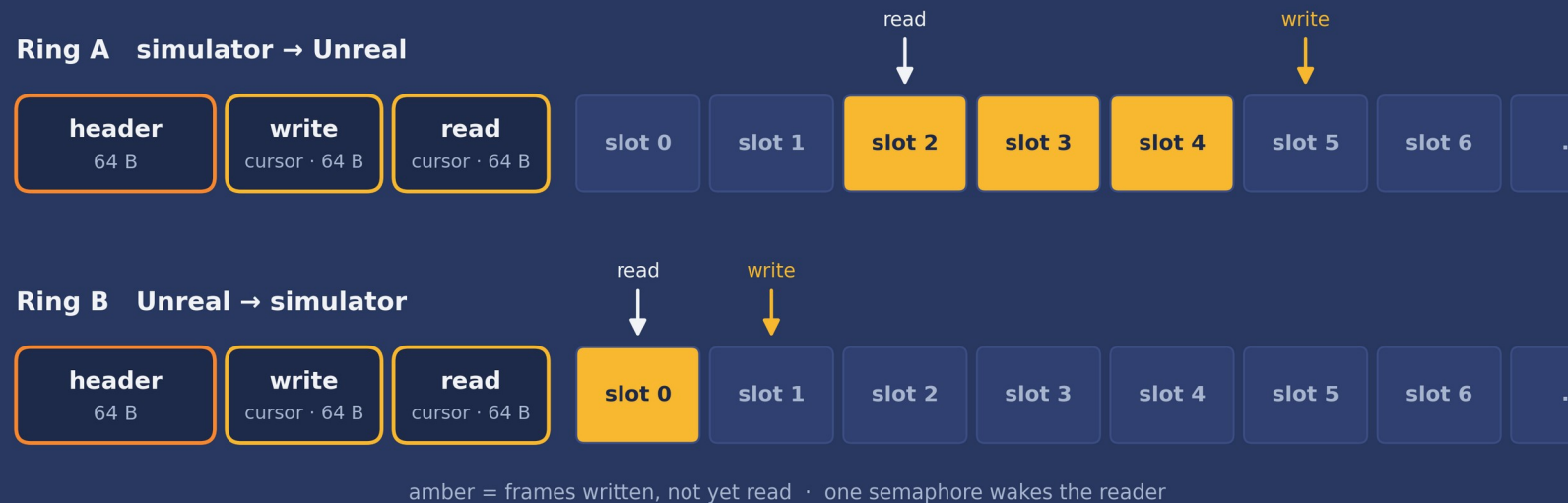
In development, and not public yet.

How it connects



What is the shm bridge?

Two ring buffers in one block of shared memory, plus a semaphore that wakes the reader.



- ▶ the simulator **owns** the region: creates it, writes both headers, removes it at shutdown
- ▶ Unreal **attaches** by name and checks the header before reading a frame
- ▶ the writer never waits: when the ring is full the frame is refused and counted

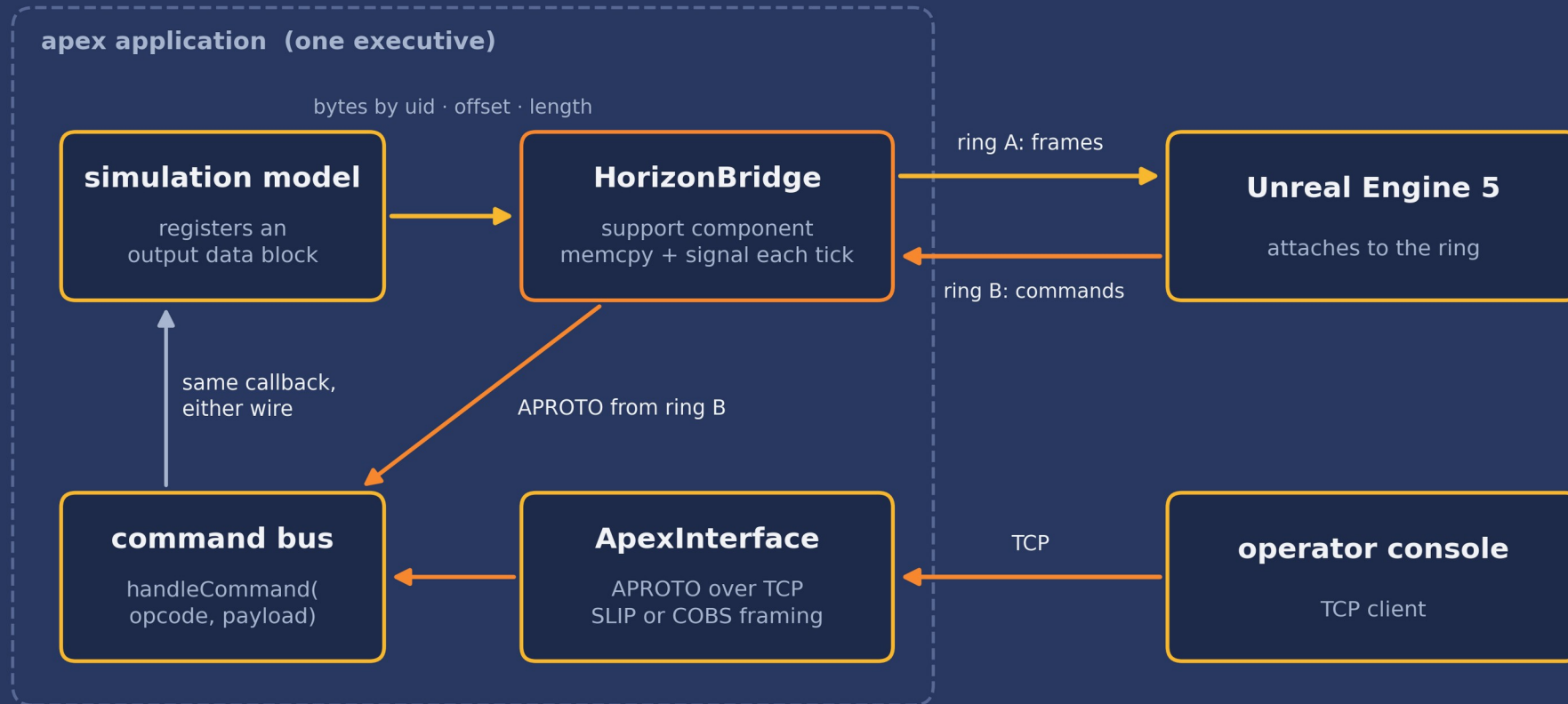
One header, both sides

```
// demos/lidar_box/contract/LidarBoxContract.hpp (abridged)
inline constexpr std::uint32_t kAppMagic    = 0x4C424F58u; // "LB0X"
inline constexpr std::uint16_t kAppVersion = 2;

struct LidarBoxFrame {                // simulator -> Unreal, 64 bytes
    float pos_x, pos_y, pos_z, yaw_rad;
    float dist_bx_pos, dist_bx_neg, dist_by_pos,
          dist_by_neg, dist_bz_pos, dist_bz_neg;
    std::uint64_t timestamp_ns;
    float box_half_x, box_half_y, box_half_z, mount_radius;
};
static_assert(sizeof(LidarBoxFrame) == 64);
```

- ▶ the simulator and the Unreal plugin both instantiate `Channel<Out, In>` with these types
- ▶ the ring header records the framework version, the app's tag and version, and the size of each payload
- ▶ a viewer built against a stale header refuses to connect: `APP_ID_MISMATCH`, `VERSION_MISMATCH`, `PAYLOAD_SIZE_MISMATCH`

The apex side: one support component



APROTO is the protocol. Shared memory and TCP are two wires into the same command bus.

Same bytes, other wires

The frames don't care how they travel, and apex already has a library for each wire.

Wire	Framing	Where it fits
Shared memory (today)	ring slot boundaries	same machine, microseconds
Unix socket	SLIP or COBS	same machine, across container boundaries
TCP	SLIP or COBS	Unreal on a different machine
UDP	one frame per datagram	high-rate state, where the newest frame wins
UART serial	SLIP or COBS, with CRC	a microcontroller or HIL board, no PC in between
CAN	split across small frames	vehicle hardware on its own bus

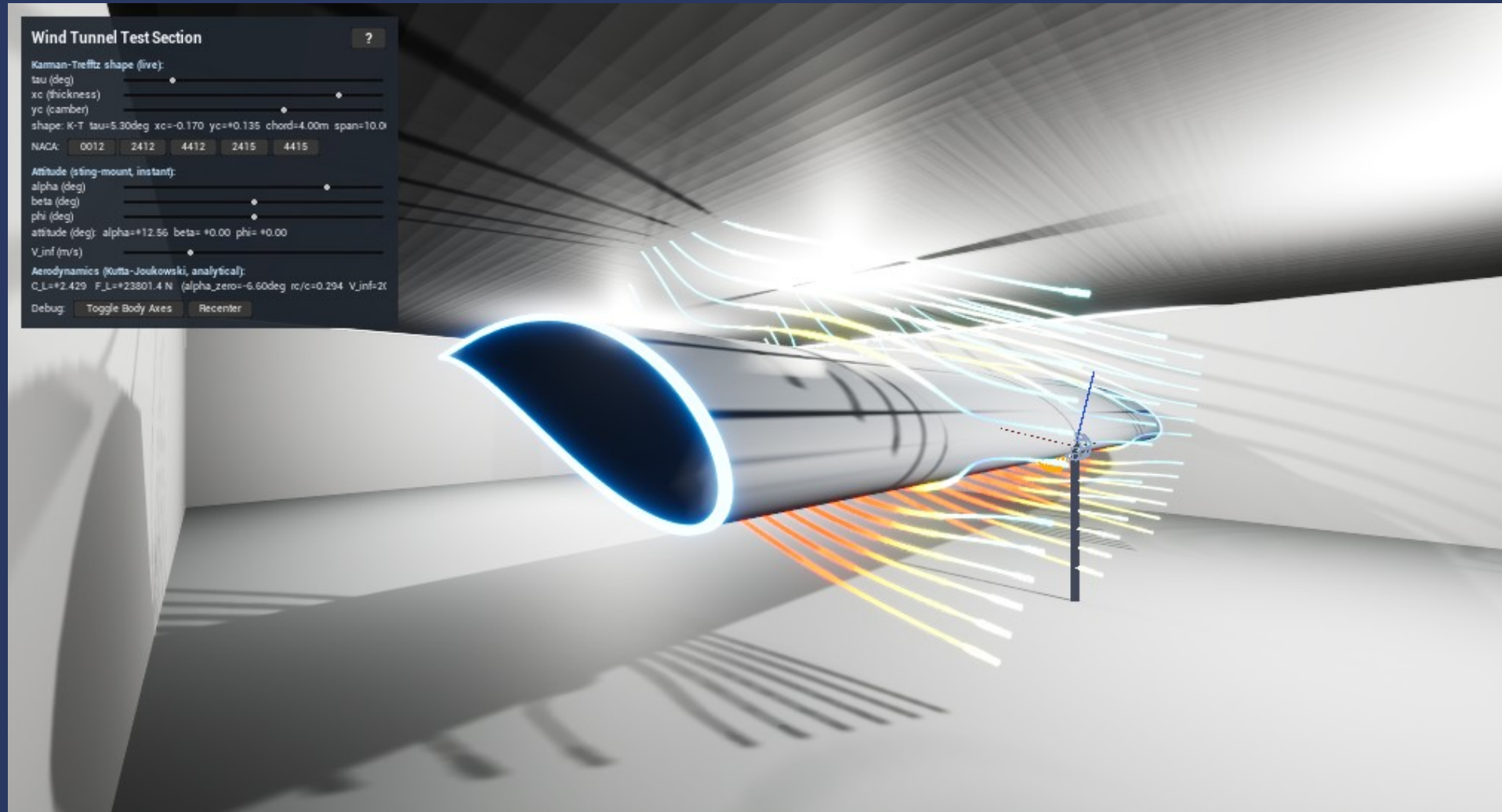
An idea, not built yet: shared memory is the only transport Horizon ships today.

The demos

Demo	What you see	Runs against
Lidar box	a sphere or drone in a lopsided box, six mounted lidar beams, every frame re-checked against a local model	a live simulator, or a built-in loopback
Airfoil tunnel	a wing section with streamlines coloured by flow speed, lift recomputed as the sliders move	standalone
Airplane tunnel	a generated airliner at any attitude, with a main-wing lift readout	standalone
Rover terrain	a rover on generated terrain, streamed ground height checked against the same terrain file	a live simulator, or a built-in loopback
Aircraft atmosphere	an airliner in a standard atmosphere, air density checked every frame, flight-mode demos on one button	a live simulator, or a built-in loopback

Every demo builds its scene in C++ at start-up and can record itself to video.

Airfoil tunnel: the live view

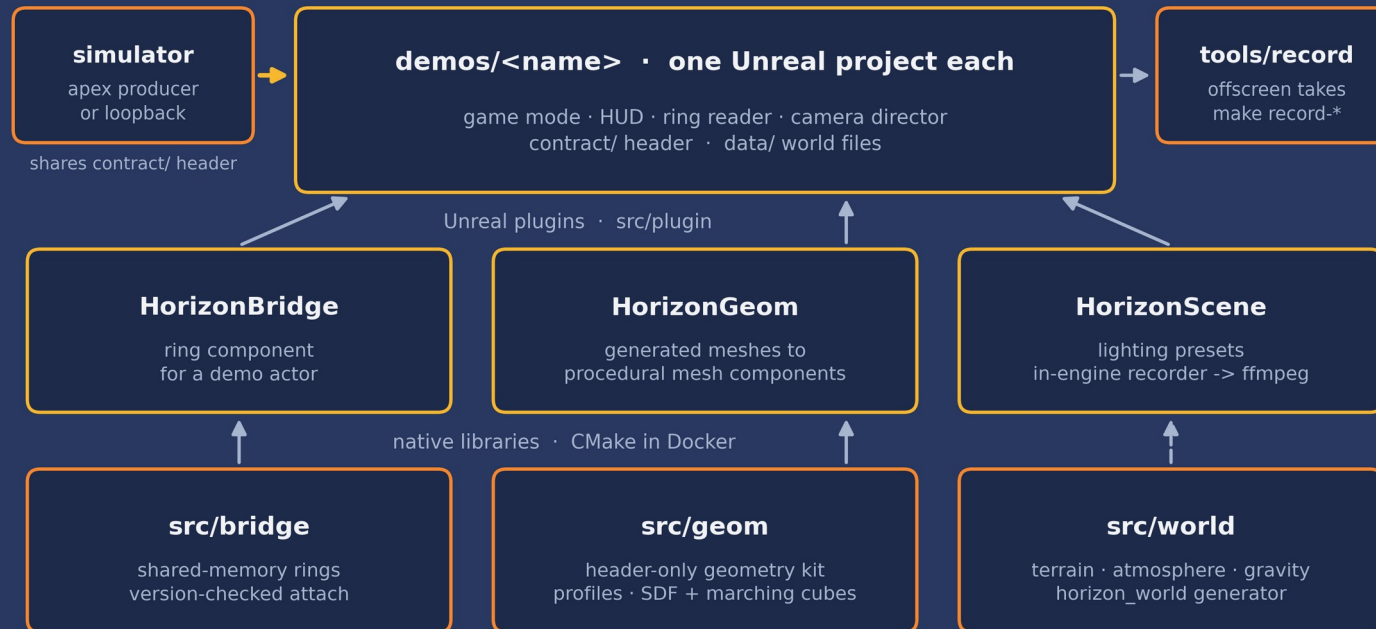


How the visuals are built

The scenes are built in C++ at start-up, on engine-default render settings.

Technique	What it does
Models	the airliner is NASA's public-domain 3D model; the rover, drone and wings come from an experimental C++ geometry generator
Materials authored in code	terrain shaded by slope, with noise grain that fades with distance so it never sparkles
Glow is real geometry	lidar beams and streamlines are emissive tube meshes, bright enough to bloom, so they show in every capture
Lighting presets	sky atmosphere, sun, real-time sky light, fog so beams scatter; fixed manual exposure for repeatable takes
Stream-clock motion	poses play back on the simulator's timestamps, a few frames behind, so fast vehicles don't judder
Scripted cameras and recording	a per-mode camera director, and an in-engine recorder piping frames to ffmpeg

How the repo is laid out



- ▶ **native libraries** build with CMake: the ring bridge, the geometry kit, the world file formats
- ▶ **three Unreal plugins** wrap them: bridge, geometry, scene (lighting and recorder)
- ▶ **each demo** is its own Unreal project with a contract header shared with the simulator

SECTION 6 · ZENITH

Operating it

One operations console for every target

What is Zenith?

The ground side of a flight system: one console to watch and command what is running.

- ▶ live telemetry charts, history, and storage for every connected target
- ▶ commanding: typed command forms, one-click quick commands, file transfer
- ▶ apex, NASA **cFS**, and **F-prime** side by side in the same console
- ▶ everything target-specific comes from **generated files**, not code in Zenith
- ▶ Rust backend, React UI, SQLite storage, one Docker container

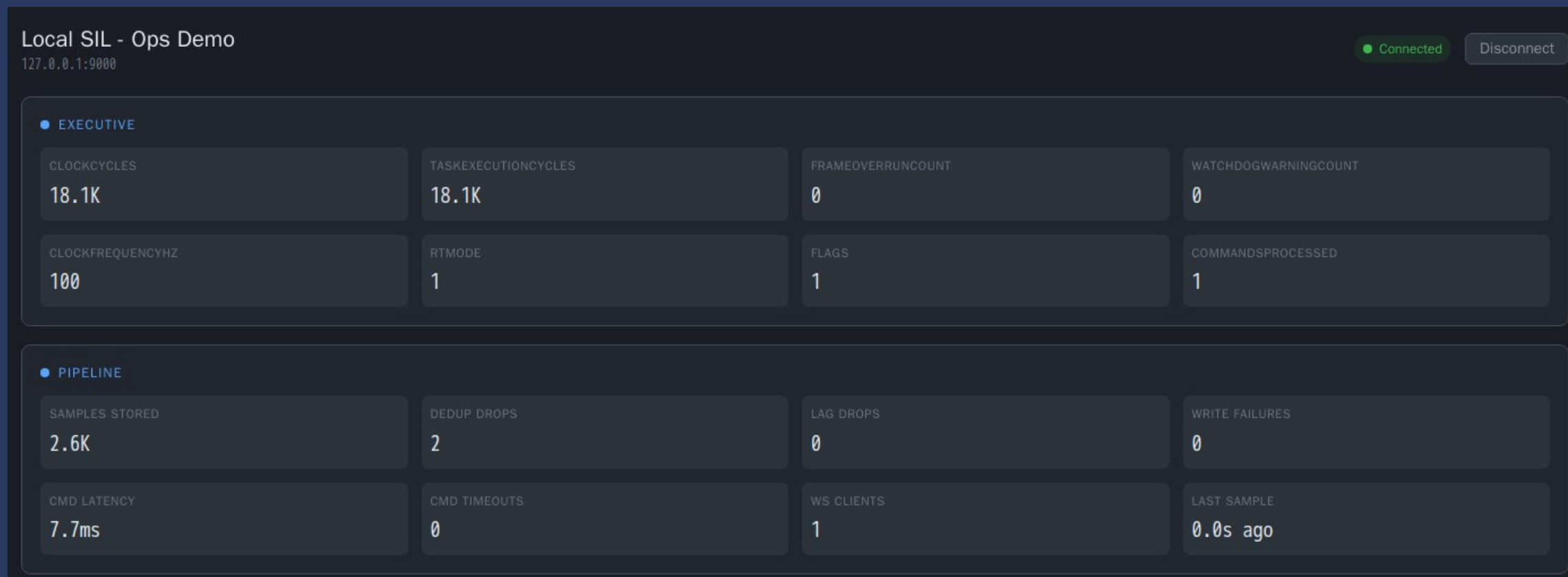
One engine, three frameworks

Each target is one entry in a config file. The wire is described, not coded.

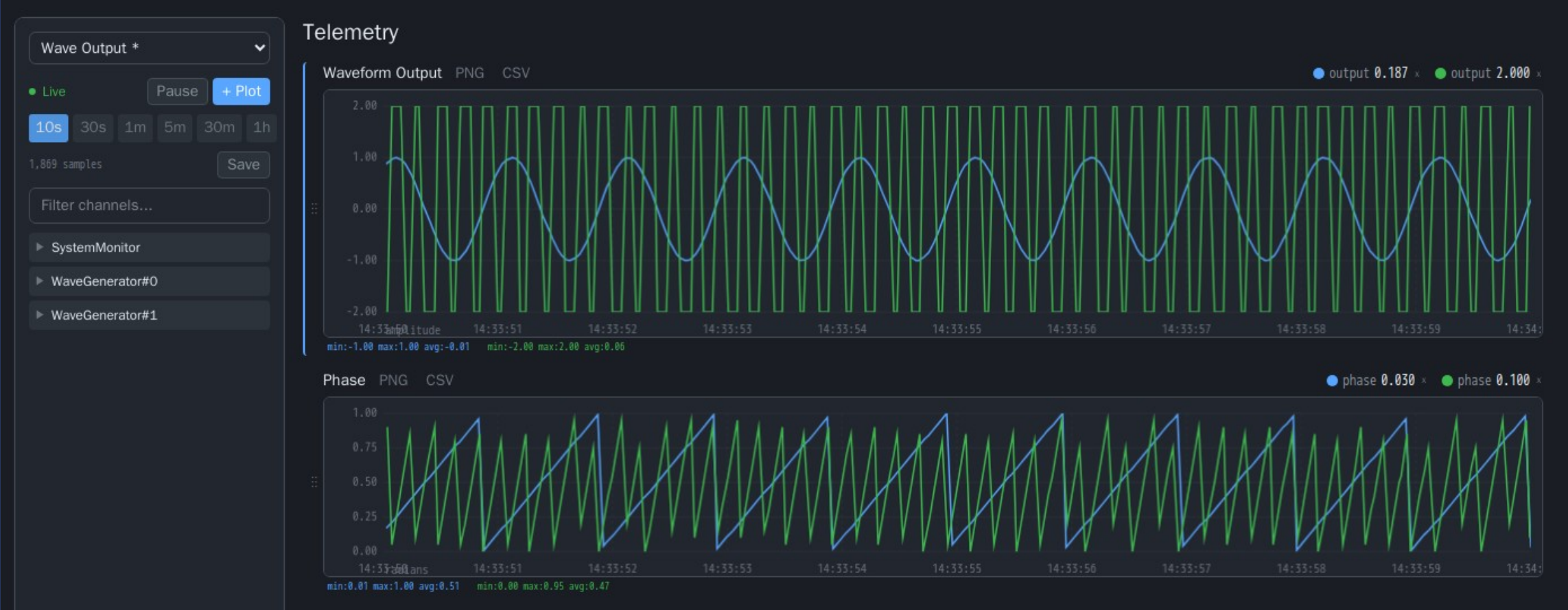
Framework	Wire	Link	Dictionary source
apex	SLIP-framed APROTO	Zenith dials TCP	apex's own target generator
NASA cFS	CCSDS space packets	UDP, downlink armed on connect	the flight binary's debug info
F-prime	CCSDS TM frames	the vehicle dials in	F-prime's JSON dictionary

A new framework is a generator plus config, not a change to the engine.

Dashboard: health at a glance



Telemetry: live strip charts



Commanding: one click to a running vehicle

Commanding

QUICK COMMANDS

NOOP

Ping

Pause

Resume

Sleep

Wake

Seq 1: north 10 m

Seq 2: north + east

Seq 3: legs + strobe

Seq 5: 30 m square

Seq 6: HALT

Seq 7: resume

Seq 9: home

Cut lidar

Restore lidar

SEND COMMAND

COMPONENT

COMMAND

Action (0x000500)

NOOP (0x0000)

Connectivity check

Send

Pin

Raw Mode

HISTORY (0)

No commands sent yet.

6 · Visualization and operations: Zenith

121

SECTION 7

Live demos

Simulation in Apex CSF, drawn in Unreal, operated from Zenith

Coming up: the horizon demo

Everything so far in one place: simulations built on the runtime, drawn live in Unreal, operated from Zenith, and in the newest scenario, controlled from a microcontroller.

Scenario	In Apex CSF	In Unreal
Lidar box	a body drifting inside a box with six lidar beams, 50 Hz	the box and beams, rebuilt from the stream
Rover on terrain	a rover with terrain lidar, slope and slip	terrain meshed from the same tile file
Aircraft in the atmosphere	a 6DOF four-engine transport, six-loop autopilot, turbulence, flight-mode sequences	a procedural airliner
Rover on a microcontroller	its control law on a NUCLEO-F767ZI, lidar on its own serial wire, safety halts, a Zenith checklist	the HUD and a command panel

- ▶ **Shared world data:** Apex CSF and Unreal load the same terrain and atmosphere files, and check every frame that they agree.
- ▶ **Frames out, commands back** over shared memory; sequences and watchpoints drive each scenario.

LIVE DEMO 1

Visualizing drone lidar

Apex CSF computes the motion and six lidar ranges; Unreal draws them

Drone lidar: the configuration

Apex CSF -- Lidar box: apex computes, Unreal renders

the first apex-connected Unreal demo; no microcontroller

apex producer (lidar_box)

laptop, Docker, 50 Hz

LidarBoxProducer

sw model

3D Lissajous drift + steady yaw (closed form, dt 20 ms)

BoxClearanceLidar

sensor

6 body-mounted pods, slab ray-to-wall, optional Gaussian noise

Scene tunables

tprm

room 8 x 6 x 5 m, mount radius 0.5 m (streamed every frame)

ShmRingBridge

support

50 Hz LBOX/v2 64 B -> /lidar_box (egress only)

pose + 6 ranges + scene

64 B @ 50 Hz

Unreal Engine 5 (horizon)

same laptop

renders

room from the streamed scene; sphere OR procedural drone (toggle)

beams

six emissive beams, pod tip to the measured wall point

check 2

recomputes all six ranges from each frame's pose: MATCH

check 4

physics traces against the wall meshes land on the same points

loopback

feeds itself synthetic frames when no producer is running

check 1: byte layout static_asserted on both sides

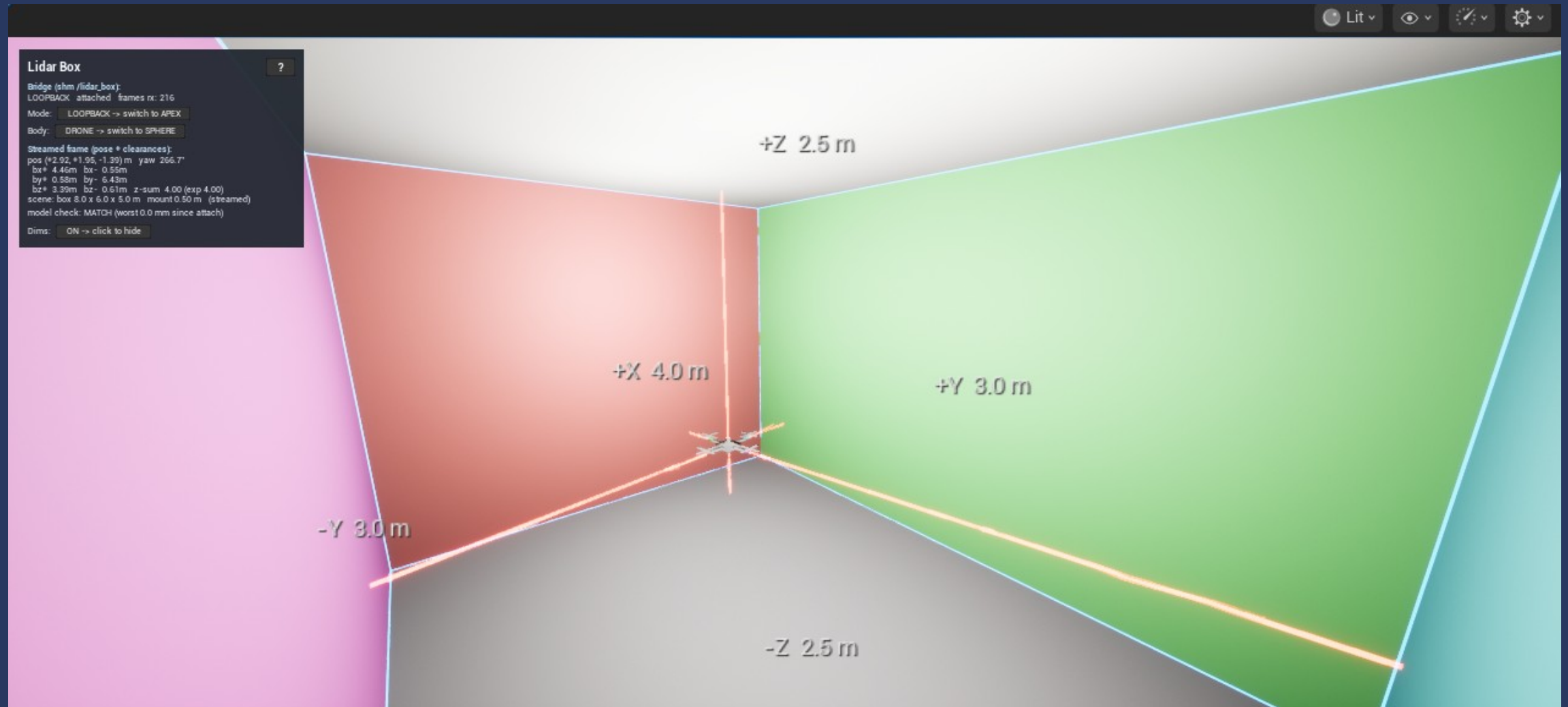
check 3: hand-derived golden vectors in both test suites

Drone lidar: the model and the checks

- ▶ **Motion**: a closed-form 3D Lissajous drift plus a steady yaw, stepped every 20 ms, so the path never closes and all six ranges keep changing.
- ▶ **Sensor**: six pods on a 0.5 m ring, each ranged to the wall in closed form; optional seeded noise, off for the demo.

Check	Catches
Frame layout asserted on both sides	byte drift between the two code bases
Unreal recomputes all six ranges live	either side's math drifting
Hand-derived golden vectors in both test suites	both sides being wrong the same way
Unreal's physics traces hit the same wall points	errors a formula and its copy would share

Drone lidar, live

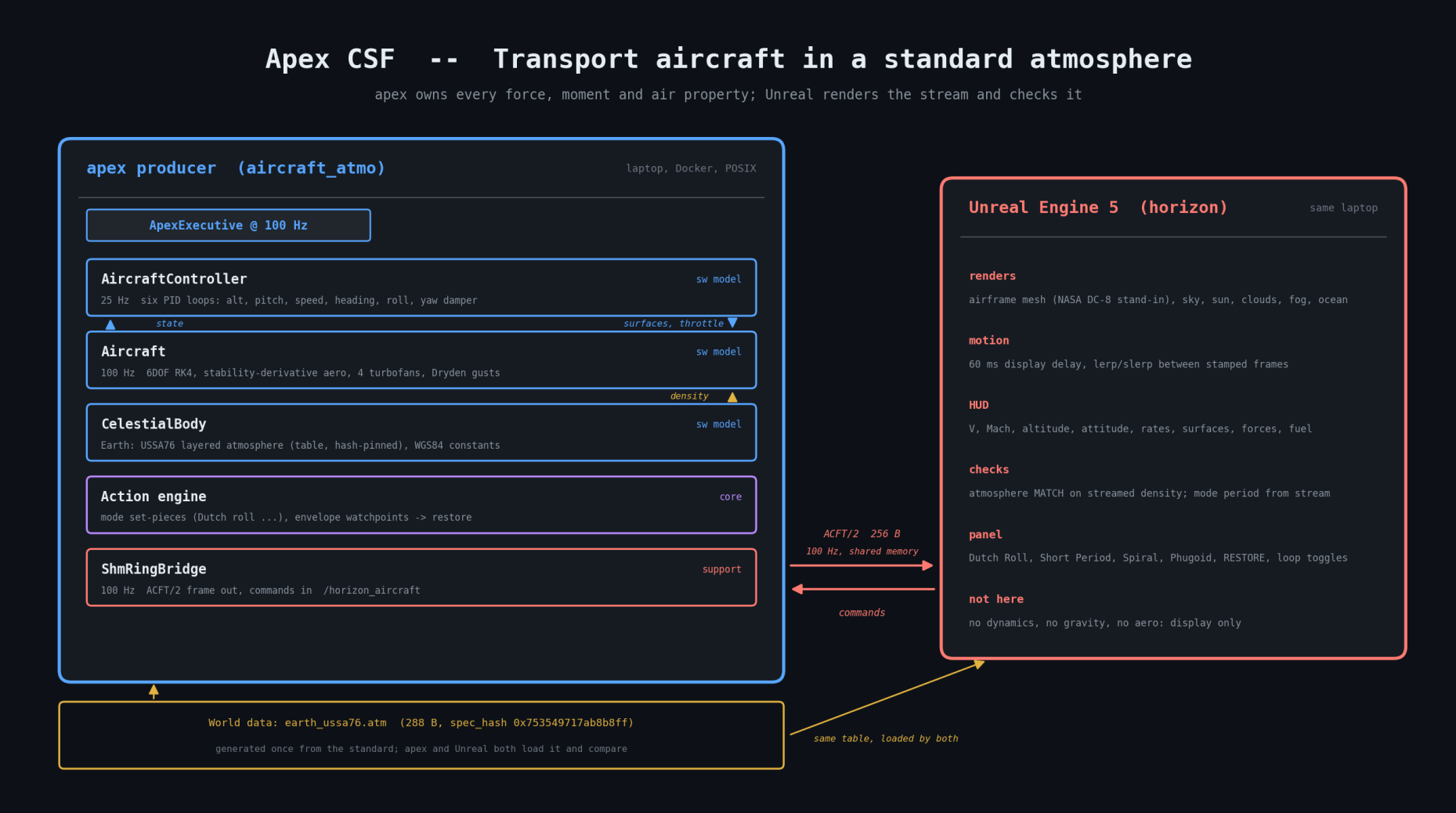


LIVE DEMO 2

Flying a transport aircraft

Six-degree-of-freedom flight under autopilot, and a Dutch roll on demand

Transport aircraft: the configuration

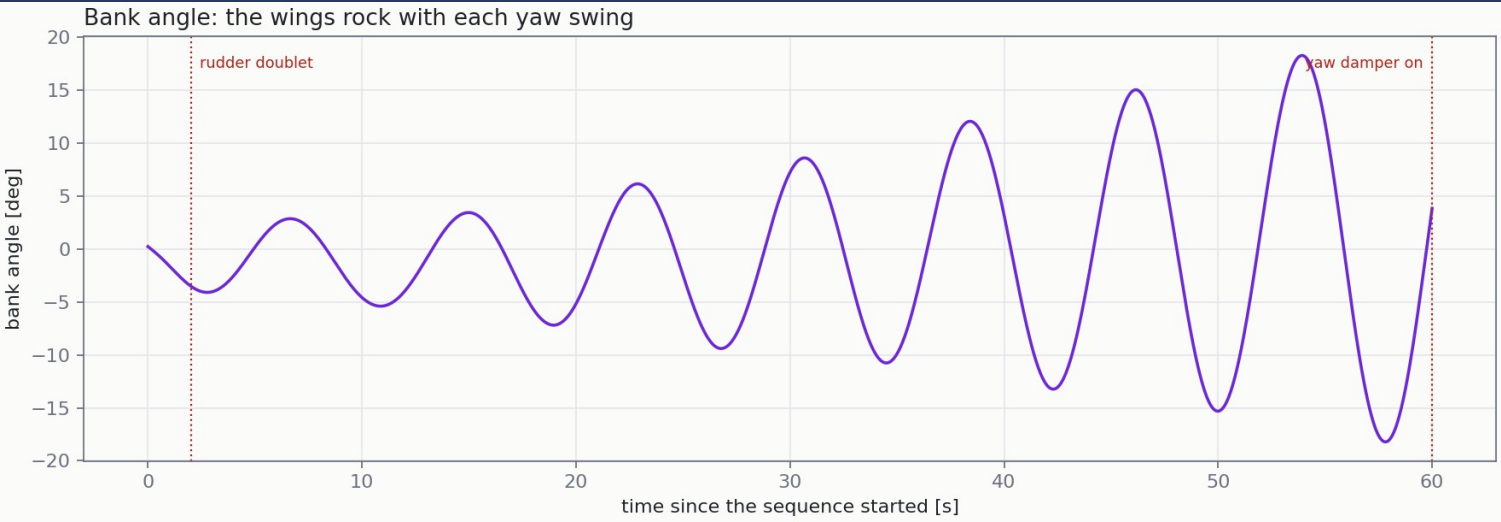
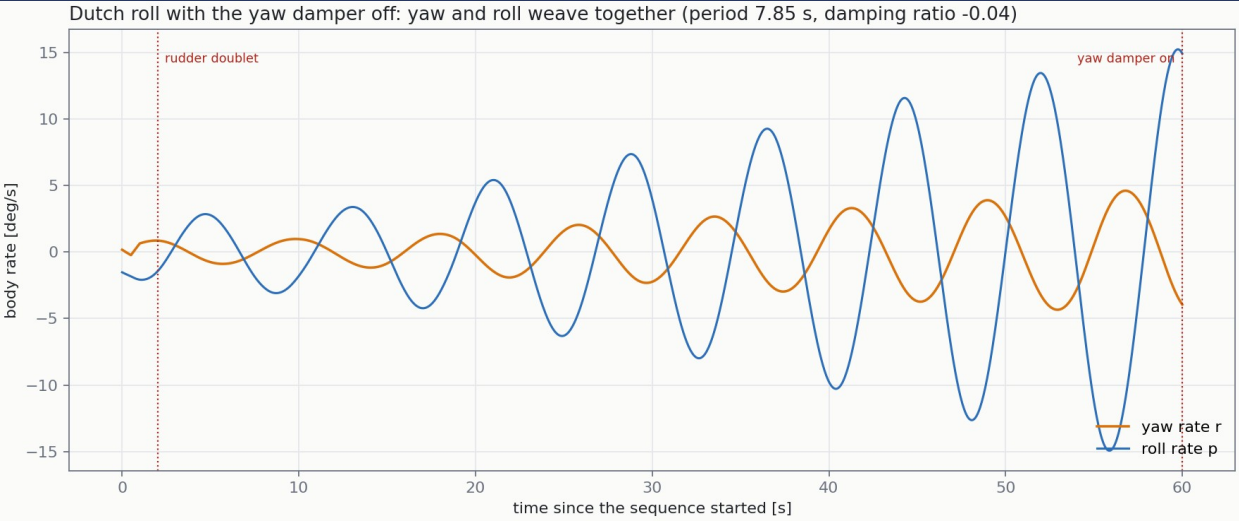


Transport aircraft: the models

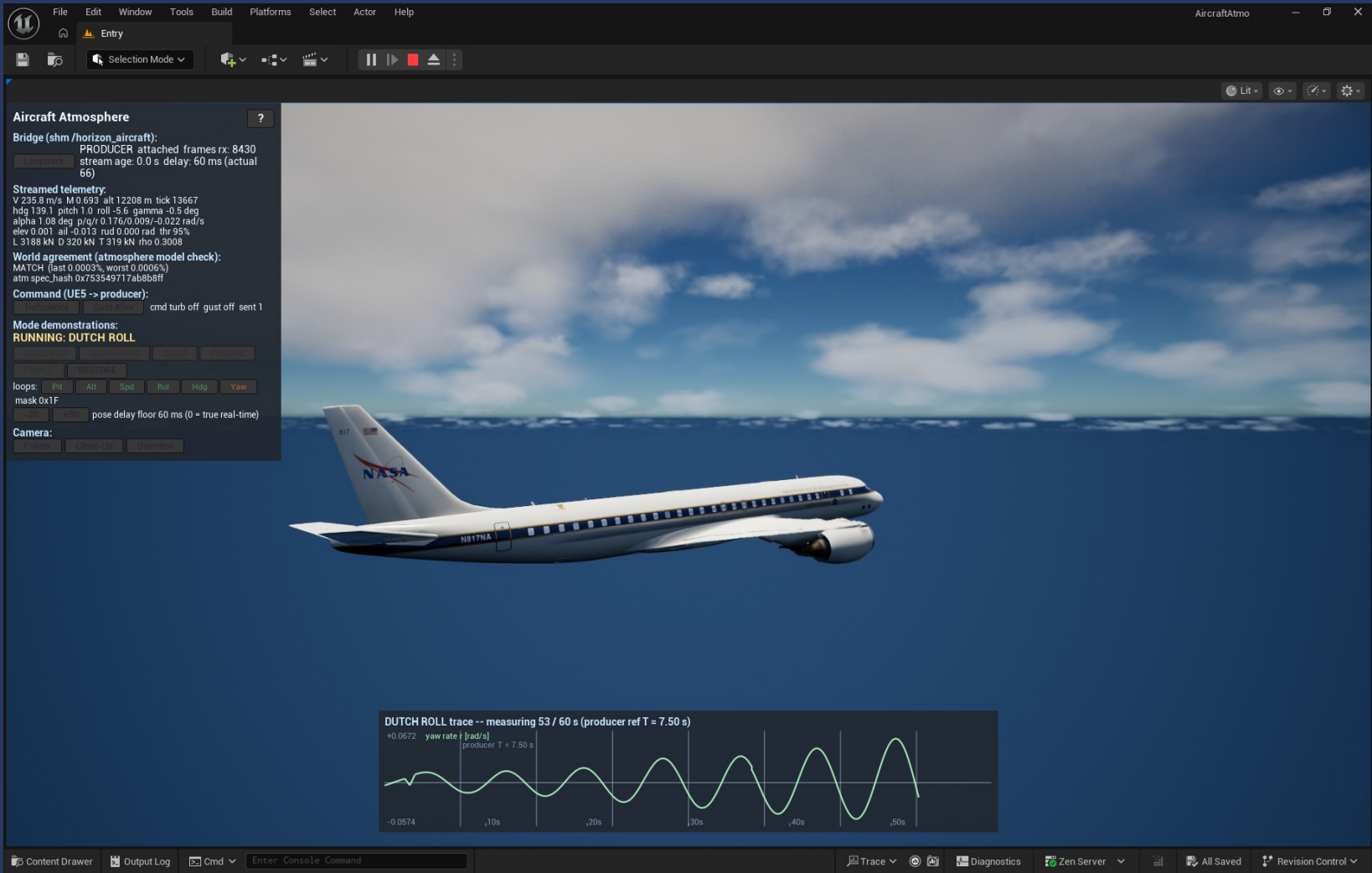
Model	How	Key numbers
Rigid body	13 states, quaternion attitude, RK4	10 ms step; 304 t at the start, 142 t of it fuel
Aerodynamics	linear stability derivatives about cruise	about 30 derivatives
Engines	four two-spool turbofans; thrust lapses with density	250 kN each at sea level
Atmosphere	US Standard Atmosphere 1976 table, pinned by hash	0.3016 kg/m ³ at 12,192 m
Turbulence	Dryden gust filters	light: 1.0 m/s horizontal, 0.5 m/s vertical
Autopilot	cascaded PID loops and a yaw damper	25 Hz; bank limit 25°

- Envelope watchpoints on speed, bank and altitude fire a sequence that re-engages every loop.

The Dutch roll, measured



Transport aircraft, live

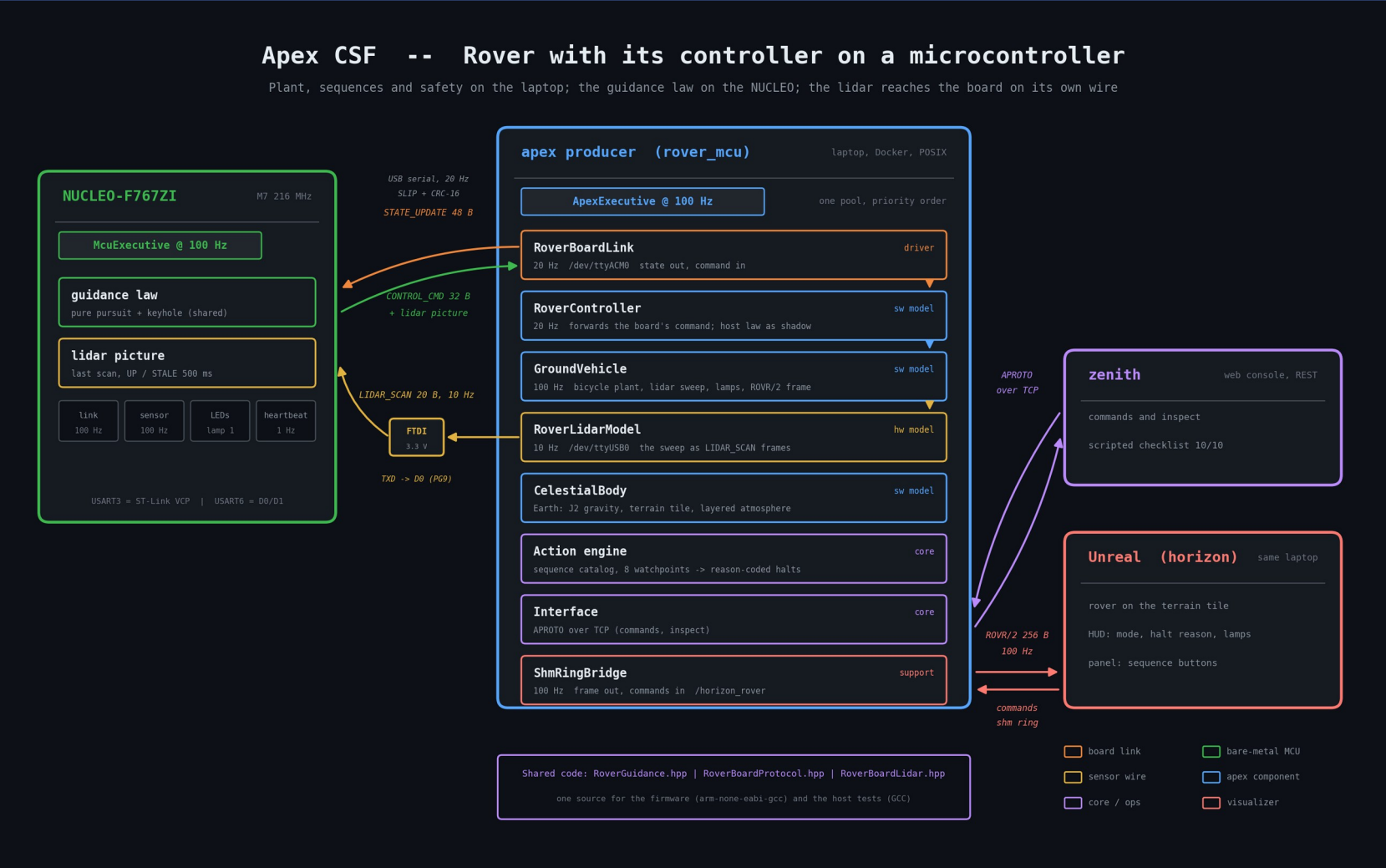


LIVE DEMO 3

Rover on a microcontroller

The guidance law on a NUCLEO-F767ZI, lidar on its own wire, operated from Zenith

Rover on a microcontroller: the configuration



Rover on a microcontroller: the rig



Rover: the models and their fidelity

Part	What is modeled	Fidelity
Vehicle motion	a bicycle model: 1.5 m wheelbase, 33° steering lock, speed toward the throttle target	kinematic; no tire forces or suspension
Terrain	height, slope and a slip flag from the shared terrain tile	height map only
Lidar geometry	8 flat rays across 90°, marched to 500 m, 10 Hz	terrain only; ranges quantized to 5 m
Lidar sensor	each sweep sent as a real serial frame on its own wire	real framing and timeouts; no noise
Controller	the guidance law on the NUCLEO-F767ZI at 20 Hz	the production code path
Board link	USB serial, CRC, timeouts, reboots	real

Rover: what reached the board



- **The board's closest return matched the plant's** in 1,867 of 1,915 samples, and was one scan behind in the rest.
- **The sensor stayed up** for the whole run: 10 scans a second over its own wire.

Rover: what I run from Zenith

#	In Zenith (target: Rover MCU)	What the audience sees
1	SET_LED on lamp 1: red, green, blue, then a white strobe	the board's LEDs follow within 50 ms, and so does the lamp in Unreal
2	Seq 2: north + east, on the Drive layout	the board steers; its command overlays the laptop's shadow copy
3	Lidar layout, then Seq 9: home	the scan counter climbing at 10 a second, hit bits changing as it turns
4	Seq 5: 30 m square, then Seq 6: HALT partway along a leg	the tour is cancelled; lamps flash red; the HUD says manual halt
5	SET_LED while halted	EXEC_FAILED in the history; the lamps keep flashing red
6	Seq 7: resume, then Cut lidar	sensor STALE and a sensor-lost halt within 2 s
7	Restore lidar, Seq 7: resume, Seq 9: home	the halt holds until acknowledged; the rover parks at the anchor

Rover on a microcontroller, live

Rover Terrain ?

Bridge (shm /horizon_rover):
PRODUCER attached frames rx: 513 (100 Hz)

Streamed frame:
pos (39.500000, -105.500000) deg alt -1781.00 m
heading 0.0 deg speed 0.00 m/s tick 8715
ground -1781.00 m slope 3.8 deg @ 274 deg
slip 0 off-terrain 0 lidar rays 8

World agreement (elevation model check):
MATCH (last 0.000 m, worst 0.000 m)
terrain spec_hash 0xe64871f7c9977289

Camera:
[Front] [Back Left] [Top] [Overview]

Command (UE5 -> producer):
[Stop] [Resume] ● 100% sent: 0

Board (controller on the MCU):
link UP / tick advancing board tick 1743 load 0%
mode HOLD | last cmd: none
board lidar UP / scans advancing nearest 35 m hits []

Sequence:
idle | lamp 1 OFF solid lamp 2 OFF solid

[Seq 1] [Seq 2] [Seq 3] [Seq 4] [Seq 5]
[HALT] [RESUME (?)] [HOME (?)] [W.B.]
[Start id]
[Mode HOLD] [Mode TRAJECTORY] [410 MIN]
[410 MIN] [Get 0.0]
[Lamp 1] [GREEN] [10 Hz] [Send LED]
[Send LED Lamp 2]

